

PIANIFICAZIONE E CONTROLLO DI UN ROBOT MOBILE DIFFERENZIALE

Documentazione tecnica di progetto

*Modello cinematico differenziale · Path planning (Cell Decomposition, PRM, RRT, APF) · Controllo $\rho-\alpha$
· Simulazione e animazione in MATLAB*

Voce	Contenuto
Ateneo	Università San Raffaele Roma
Studente & Matricola	Daniele Murgese (0322500069)
Ambiente	MATLAB R2016b o successivo (compatibile con GNU Octave 6+)
Toolbox	Nessuno: grafo, Dijkstra/A* e collision checking sono implementati nel progetto
Punto di ingresso	<code>main.m</code> (esecuzione completa), <code>run_all_tests.m</code> (verifiche automatiche)
Parametri	<code>default_params.m</code> – tutti i valori numerici sono configurabili dall'utente
Versione documento	1.0 – ottobre 2026
Link Repository	https://github.com/danielemurgese992/Robotica-intelligente

Indice

1. Introduzione e obiettivi	5
1.1 Scelte progettuali principali	5
1.2 Corrispondenza tra traccia e implementazione	5
2. Struttura del progetto	7
2.1 Flusso di esecuzione	7
3. Fondamenti teorici	9
3.1 Modello cinematico del robot differenziale	9
3.2 Rappresentazione dell'ambiente e degli ostacoli	9
3.3 Cell decomposition con retraction	10
3.4 Metodi probabilistici: PRM e RRT	10
3.5 Artificial Potential Fields	11
3.6 Controllo cinematico del robot	12
3.7 Indicatori di prestazione	13
4. Descrizione passo dopo passo dell'implementazione	14
Passo 1 – Definizione dei parametri	14
Passo 2 – Costruzione e verifica dell'ambiente	14
Passo 3 – Pianificazione	14
Passo 4 – Post-elaborazione e waypoint	14
Passo 5 – Simulazione in anello chiuso	14
Passo 6 – Analisi	14
Passo 7 – Visualizzazione	15
5. Parametri configurabili	16
5.1 Robot (p.robot)	16
5.2 Ambiente (p.env)	16
5.3 Configurazioni iniziale e finale (p.q0, p.qf)	16
5.4 Pianificazione	16
5.5 Cell decomposition (p.cell)	17
5.6 PRM (p.prm)	17
5.7 RRT (p.rrt)	17
5.8 Artificial Potential Fields (p.apf)	17
5.9 Post-elaborazione del percorso (p.post)	18
5.10 Controllo (p.ctrl)	18
5.11 Simulazione (p.sim)	18
5.12 Visualizzazione (p.viz)	19
5.13 Definire uno scenario personalizzato	19
5.14 Indicazioni per la scelta dei valori	19

6. Installazione, esecuzione e test	21
6.1 Requisiti	21
6.2 Avvio rapido	21
6.3 Uso da Command Window	21
6.4 Test automatici	22
6.5 Confronto dei pianificatori	22
6.6 Problemi comuni	22
7. Risultati delle simulazioni	24
7.1 Traiettoria pianificata ed eseguita (PRM, scenario base)	24
7.2 Errore di tracking	25
7.3 Velocità delle ruote	25
7.4 Struttura dei pianificatori	26
7.5 Campi di potenziale e minimi locali	27
8. Analisi dei risultati	29
8.1 Confronto dei pianificatori	29
8.2 Effetto dei guadagni del controllore	29
8.3 Integrazione numerica, rumore e variante della legge di controllo	30
8.4 Considerazioni conclusive	30
Appendice A – Codice sorgente completo	31
A.1 main.m	31
A.2 run_project.m	32
A.3 README.md	32
A.4 config/default_params.m	33
A.5 config/scenario_library.m	35
A.6 config/validate_params.m	36
A.7 environment/build_environment.m	37
A.8 environment/clearance.m	37
A.9 environment/is_free.m	37
A.10 environment/is_path_free.m	38
A.11 environment/is_segment_free.m	38
A.12 environment/make_obstacle.m	38
A.13 environment/obstacle_distances.m	38
A.14 model/diffdrive_dynamics.m	39
A.15 model/integrate_step.m	39
A.16 model/saturate_wheels.m	40
A.17 model/unicycle_to_wheels.m	40
A.18 model/wheels_to_unicycle.m	40
A.19 planning/apf_potential.m	41

A.20	planning/graph_search.m	42
A.21	planning/plan_apf.m	42
A.22	planning/plan_cell_decomposition.m	44
A.23	planning/plan_path.m	45
A.24	planning/plan_prm.m	45
A.25	planning/plan_rrt.m	46
A.26	planning/resample_path.m	47
A.27	planning/shortcut_path.m	47
A.28	control/controller_rho_alpha.m	48
A.29	control/point_on_path.m	48
A.30	control/simulate_robot.m	48
A.31	analysis/compare_planners.m	50
A.32	analysis/compute_metrics.m	50
A.33	analysis/point_to_path_distance.m	51
A.34	analysis/print_metrics.m	51
A.35	visualization/animate_robot.m	52
A.36	visualization/plot_environment.m	52
A.37	visualization/plot_planner_debug.m	53
A.38	visualization/plot_pose.m	54
A.39	visualization/plot_results.m	54
A.40	visualization/robot_shape.m	55
A.41	utils/path_length.m	56
A.42	utils/set_seed.m	56
A.43	utils/wrap_to_pi.m	56
A.44	tests/run_all_tests.m	56

1. Introduzione e obiettivi

Il progetto realizza in MATLAB un sistema completo per un **robot mobile differenziale** che si muove nel piano XY con due ruote motrici indipendenti. Come richiesto dalla traccia, il sistema copre quattro aspetti:

1. **modellazione cinematica** del robot (modello differenziale, cinematica diretta e inversa delle ruote);
2. **pianificazione del moto** in un ambiente con ostacoli circolari e poligonali;
3. **controllo del moto** lungo il percorso pianificato con un controllore cinematico $v = k_p \cdot p$, $\omega = k_d \cdot \dot{p}$;
4. **simulazione e visualizzazione**: traiettoria pianificata ed eseguita, errore di tracking, velocità delle ruote, animazione.

La traccia richiede almeno un metodo di pianificazione: il progetto ne implementa **quattro**, selezionabili con un solo parametro (`p.planner`): Cell Decomposition con retraction (§4.1 della traccia), PRM e RRT (§4.2) e Artificial Potential Fields con discussione e gestione dei minimi locali (§4.3). Questo permette anche un confronto quantitativo dei metodi sullo stesso scenario (Capitolo 8).

Il robot si muove **esclusivamente con il modello differenziale**: il controllore produce (v, ω) , che vengono convertiti in velocità delle ruote (ω_r, ω_l) , saturati come farebbero dei motori reali, e solo le velocità delle ruote entrano nell'integratore dello stato. In questo modo ogni traiettoria eseguita è fisicamente realizzabile dal veicolo.

1.1 Scelte progettuali principali

- **Completamente parametrico.** Nessun valore numerico è cablato nel codice: robot, ambiente, configurazioni q_0/q_f , pianificatori, controllore, simulazione e grafica dipendono da una sola struttura `p` creata da `default_params.m`. L'utente sceglie i valori in `main.m` o da Command Window.
- **Nessun toolbox richiesto.** Grafo, Dijkstra/A*, distanze geometriche e collision checking sono implementati da zero: il codice gira su MATLAB base (R2016b o successivo) ed è stato verificato anche con GNU Octave 8.4.
- **Modularità.** Ogni funzione ha una sola responsabilità ed è organizzata in cartelle (config, model, environment, planning, control, analysis, visualization, utils, tests).
- **Verificabilità.** Una suite di 18 test automatici controlla il modello cinematico (con soluzioni analitiche note), la geometria, i quattro pianificatori, i minimi locali di APF e il controllo in anello chiuso.

1.2 Corrispondenza tra traccia e implementazione

Richiesta della traccia	Dove è implementata
§2 Modello cinematico differenziale	model/wheels_to_unicycle.m, diffdrive_dynamics.m, integrate_step.m
§3 Ambiente, ostacoli circolari/poligonali, q_0 , q_f	config/scenario_library.m, environment/*.m, plot_environment.m
§4.1 Cell decomposition + adiacenza + retraction + Dijkstra	planning/plan_cell_decomposition.m, graph_search.m
§4.2 PRM / RRT	planning/plan_prm.m, planning/plan_rrt.m

Richiesta della traccia	Dove è implementata
§4.3 APF, gradiente, minimi locali	planning/apf_potential.m, planning/plan_apf.m
§5.1 Percorso $\rightarrow$ waypoint o riferimento continuo	planning/resample_path.m, p.ctrl.ref_mode
§5.2 Controllore $v = k_p \rho$, $\omega = k_\alpha \alpha$	control/controller_rho_alpha.m
§5.3 Conversione $(v, \omega) \rightarrow (\omega_R, \omega_L)$	model/unicycle_to_wheels.m, saturate_wheels.m
§5.4 Integrazione numerica	model/integrate_step.m (Eulero, RK4, esatta), control/simulate_robot.m
§6 Traiettorie, errore, velocità ruote, animazione	visualization/plot_results.m, animate_robot.m
§7 Analisi dei risultati	analysis/compute_metrics.m, compare_planners.m, Capitolo 8

2. Struttura del progetto

Il progetto è contenuto nella cartella **RobotMobileDifferenziale**. Il file **main.m** è il punto d'ingresso; **run_project.m** esegue la pipeline completa e restituisce tutti i risultati in una struttura.

RobotMobileDifferenziale/	
— main.m	punto d'ingresso: parametri utente + esecuzione
— run_project.m	pipeline: ambiente → piano → controllo → analisi → grafici
— README.md	
— config/	
— default_params.m	TUTTI i parametri con valori di default
— validate_params.m	controlli di coerenza dei parametri
— scenario_library.m	scenari predefiniti: base, narrow, trap
— model/	
— wheels_to_unicycle.m	$(\omega_R, \omega_L) \rightarrow (v, \omega)$
— unicycle_to_wheels.m	$(v, \omega) \rightarrow (\omega_R, \omega_L)$
— diffdrive_dynamics.m	$\dot{q} = f(q, \omega_R, \omega_L)$
— integrate_step.m	Eulero / RK4 / soluzione esatta
— saturate_wheels.m	saturazione che preserva la curvatura
— environment/	
— make_obstacle.m	creazione ostacoli circolari e poligonali
— build_environment.m	costruzione ambiente da parametri
— obstacle_distances.m	distanza con segno + gradiente
— clearance.m, is_free.m, is_segment_free.m, is_path_free.m	
— planning/	
— plan_path.m	selezione del pianificatore + post-elaborazione
— plan_cell_decomposition.m, plan_prm.m, plan_rrt.m, plan_apf.m	
— apf_potential.m	potenziale totale e gradiente
— graph_search.m	Dijkstra / A*
— shortcut_path.m	semplificazione collision-free
— resample_path.m	generazione dei waypoint
— control/	
— controller_rho_alpha.m	$v = k_p p, \omega = k_\alpha \alpha$
— simulate_robot.m	anello chiuso con il modello differenziale
— point_on_path.m	riferimento continuo lungo il percorso
— analysis/	
— compute_metrics.m, print_metrics.m	
— point_to_path_distance.m	errore di tracking laterale
— compare_planners.m	confronto quantitativo dei metodi
— visualization/	
— plot_environment.m, plot_pose.m, plot_planner_debug.m	
— plot_results.m	grafici richiesti dalla traccia
— robot_shape.m, animate_robot.m	animazione (+ GIF opzionale)
— utils/	wrap_to_pi.m, path_length.m, set_seed.m
— tests/	run_all_tests.m 18 test automatici

2.1 Flusso di esecuzione

La funzione **run_project(p)** esegue in sequenza i passi seguenti; ciascuno è descritto in dettaglio nel Capitolo 4.

1. **build_environment** costruisce ostacoli, bordi e configurazioni q_0, q_f dallo scenario scelto;
2. **validate_params** controlla i parametri e verifica che q_0 e q_f siano libere;
3. **plan_path** esegue il pianificatore scelto, semplifica il percorso e genera i waypoint;
4. **simulate_robot** chiude l'anello controllore → ruote → modello differenziale;
5. **compute_metrics** calcola gli indicatori (errore di tracking, clearance, tempi, saturazioni);
6. **plot_planner_debug**, **plot_results** e **animate_robot** producono grafici e animazione.

3. Fondamenti teorici

3.1 Modello cinematico del robot differenziale

Lo stato del robot è la configurazione nel piano $q = [x \ y \ \theta]^T$, dove (x, y) è il punto medio dell'asse delle ruote e θ l'orientamento rispetto all'asse x . Gli ingressi sono le velocità angolari delle ruote destra ω_R e sinistra ω_L . Indicando con r il raggio delle ruote e con d la distanza tra di esse (carreggiata), le velocità lineari dei punti di contatto sono $r \cdot \omega_R$ e $r \cdot \omega_L$. La velocità del punto medio e la velocità di rotazione del corpo sono:

$$v = (r/2) \cdot (\omega_R + \omega_L) \quad \omega = (r/d) \cdot (\omega_R - \omega_L)$$

Il vincolo di rotolamento senza strisciamento impedisce il moto laterale (vincolo **anolonomo** $\dot{x} \sin \theta - \dot{y} \cos \theta = 0$). Le equazioni del moto sono:

$$\dot{x} = v \cos \theta \quad \dot{y} = v \sin \theta \quad \dot{\theta} = \omega$$

Il sistema ha due ingressi e tre gradi di libertà: ogni configurazione è raggiungibile (il sistema è controllabile), ma non in linea retta in qualsiasi direzione. Per questo la pianificazione produce un percorso geometrico nel piano e il controllore genera i comandi ammissibili per inseguirlo.

Cinematica inversa

Invertendo le relazioni precedenti si ottengono le velocità delle ruote che realizzano una coppia (v, ω) :

$$\omega_R = (2v + \omega d) / (2r) \quad \omega_L = (2v - \omega d) / (2r)$$

Due casi particolari, usati anche nei test automatici, chiariscono il modello: con $\omega_R = \omega_L$ il robot avanza in linea retta ($\omega = 0$) di $r \cdot \omega \cdot t$; con $\omega_R = -\omega_L$ ruota sul posto ($v = 0$) con $\theta(t) = 2r\omega \cdot t/d$.

Saturazione delle ruote

I motori hanno una velocità massima $\omega_{\max}$. Saturare indipendentemente le due ruote cambierebbe il rapporto ω_R/ω_L e quindi la curvatura del moto, portando il robot fuori dal percorso. Il progetto applica invece una **saturazione che preserva la curvatura**: se $\max(|\omega_R|, |\omega_L|) > \omega_{\max}$, entrambe le ruote sono moltiplicate per $s = \omega_{\max} / \max(|\omega_R|, |\omega_L|)$. Il robot segue lo stesso arco, solo più lentamente.

Integrazione numerica

Con ingressi costanti nel passo dt (ricostruiti dalle velocità delle ruote), lo stato è avanzato con uno dei tre metodi selezionabili tramite `p.sim.integrator`:

- **Eulero esplicito**: $q(k+1) = q(k) + dt \cdot f(q(k))$. Semplice, errore locale $O(dt^2)$.
- **Runge–Kutta del 4° ordine** (default): errore locale $O(dt^5)$, ottimo compromesso tra accuratezza e costo.
- **Soluzione esatta**: con v e ω costanti il robot percorre un arco di circonferenza di raggio v/ω , quindi $x(k+1) = x + (v/\omega)[\sin(\theta + \omega dt) - \sin \theta]$, $y(k+1) = y - (v/\omega)[\cos(\theta + \omega dt) - \cos \theta]$, $\theta(k+1) = \theta + \omega dt$ (linea retta se $\omega \approx 0$). È usata come riferimento per validare RK4.

3.2 Rappresentazione dell'ambiente e degli ostacoli

L'ambiente è un rettangolo $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ con ostacoli **circolari** (centro, raggio) o **poligonali** (vertici, anche concavi). Il robot è approssimato da un disco di raggio R ; si aggiunge un

marginale di sicurezza m . La pianificazione avviene nello **spazio delle configurazioni** della sola posizione: un punto p è libero se la distanza tra p e ogni ostacolo o bordo supera $R + m$ (ostacoli “gonfiati”, cioè i C-ostacoli). Poiché il disco è invariante per rotazione, l’orientamento non influisce sulle collisioni.

La funzione `obstacle_distances` calcola, in forma vettoriale per N punti, la **distanza con segno** da ciascun ostacolo (negativa all’interno) e il suo gradiente (versore che si allontana dall’ostacolo):

- cerchio: $d = \|p - c\| - R$, $\nabla d = (p - c) / \|p - c\|$;
- poligono: $d = \pm \min_{\text{lati}} \text{dist}(p, \text{lato})$, con segno negativo se p è interno (test `inpolygon`); $\nabla d = \pm(p - p^*) / \|p - p^*\|$ dove p^* è il punto più vicino sul bordo;
- bordi del rettangolo: distanze lineari con gradiente pari alla normale entrante.

La **clearance** è il minimo di queste distanze. Un segmento è libero se lo sono tutti i punti campionati su di esso con passo `p.post_collision_step`, scelto molto minore del margine di sicurezza.

3.3 Cell decomposition con retraction

Il metodo trasforma il problema continuo in un problema su grafo:

1. **Decomposizione.** L’ambiente è diviso in celle quadrate di lato h (`p.cell.resolution`). Una cella è *libera* se la clearance del suo centro supera $R + m$; in modalità conservativa (`p.cell.conservative = true`) si aggiunge la mezza diagonale $h\sqrt{2}/2$, così l’intera cella è garantita libera.
2. **Grafo di adiacenza.** Ogni cella libera è collegata alle celle libere adiacenti (4 o 8 vicini). Con 8 vicini un passo diagonale è ammesso solo se sono libere entrambe le celle ortogonali, per non “tagliare” lo spigolo di un ostacolo.
3. **Retraction.** Ogni cella è ridotta (retratta) al suo nodo centrale. Per avvicinare il cammino a una vera retraction sullo scheletro dello spazio libero (diagramma di Voronoi generalizzato), il costo di un arco di lunghezza L è $L \cdot (1 + w / \bar{c})$, dove $\bar{c}$ è la clearance media dei due nodi oltre il raggio gonfiato e w è `p.cell.clearance_weight`. Con $w = 0$ si ottiene il cammino più corto sulla griglia; aumentando w il cammino si allontana dagli ostacoli verso il centro dei corridoi.
4. **Ricerca.** q_0 e q_f vengono collegati alla cella libera più vicina visibile in linea retta; il cammino minimo sul grafo è trovato con **Dijkstra** (o A^* , `p.cell.search = 'astar'`, con euristica euclidea ammissibile).

Output: la sequenza di waypoint (x_i, y_i) costituita da q_0 , dai centri delle celle del cammino e da q_f . Il metodo è *risoluzione-completo*: se esiste un cammino con passaggi più larghi di circa una cella, viene trovato; passaggi più stretti richiedono una h più piccola (a costo di più celle).

3.4 Metodi probabilistici: PRM e RRT

Probabilistic RoadMap (PRM)

1. Si campionano n punti uniformi nell’ambiente e si conservano solo quelli collision-free.
2. Ogni nodo viene collegato ai suoi k vicini più prossimi entro un raggio massimo, se il segmento che li unisce è libero (local planner rettilineo).
3. q_0 e q_f sono aggiunti come nodi e il cammino minimo è cercato con Dijkstra (o A^*).
4. Se q_0 e q_f restano in componenti connesse diverse, si riprova con n moltiplicato per `p.prm.growth` (fino a `p.prm.max_attempts` volte).

Il PRM è *probabilisticamente completo*: la probabilità di non trovare un cammino esistente tende a zero all’aumentare dei campioni. È un metodo multi-query: la roadmap può essere riusata per altre coppie q_0, q_f .

Rapidly-exploring Random Tree (RRT)

L'albero parte da q_0 . A ogni iterazione si campiona un punto casuale (il goal con probabilità `goal_bias`), si individua il nodo più vicino e si avvanza verso il campione di un passo massimo δ . Il nuovo nodo è aggiunto se il segmento è libero. Quando un nodo arriva entro `goal_tol` dal goal e lo vede in linea retta, il percorso è ricostruito risalendo i genitori. L'espansione è polarizzata verso le regioni inesplorate (bias di Voronoi), il che rende RRT molto rapido negli ambienti aperti.

Post-elaborazione comune: shortcut e waypoint

I percorsi grezzi di PRM, RRT, griglia e APF contengono zig-zag o punti allineati. La funzione `shortcut_path` applica una semplificazione "string pulling": da ogni punto salta al punto più lontano raggiungibile in linea retta senza collisioni. Il percorso risultante resta collision-free per costruzione. Infine `resample_path` suddivide ciascun segmento in parti di lunghezza $\leq$ `waypoint_spacing` **mantenendo tutti i vertici**: i waypoint giacciono esattamente sul percorso pianificato, quindi non tagliano gli spigoli vicino agli ostacoli.

3.5 Artificial Potential Fields

Il robot è considerato una particella immersa in un campo di potenziale $U = U_{att} + U_{rep}$: il goal la attrae, gli ostacoli la respingono e il moto segue la direzione di massima discesa $-\nabla U$.

Potenziale attrattivo

Si usa la forma combinata quadratica/conica, con $d = \|p - p_g\|$ e soglia d^* :

$$\begin{aligned} U_{att} &= \frac{1}{2} k_a d^2 \quad (d \leq d^*) & U_{att} &= d^* k_a d - \frac{1}{2} k_a d^{*2} \quad (d > d^*) \\ \nabla U_{att} &= k_a (p - p_g) \quad (d \leq d^*) & \nabla U_{att} &= d^* k_a (p - p_g) / d \quad (d > d^*) \end{aligned}$$

La parte conica limita la forza attrattiva lontano dal goal (che altrimenti crescerebbe linearmente con la distanza), quella quadratica garantisce un arrivo dolce.

Potenziale repulsivo (Khatib)

Per ogni ostacolo e ciascun bordo, detta ρ la distanza dall'ostacolo gonfiato e ρ_0 il raggio di influenza:

$$\begin{aligned} U_{rep} &= \frac{1}{2} k_r (1/\rho - 1/\rho_0)^2 \quad (\rho \leq \rho_0), \quad 0 \text{ altrimenti} \\ \nabla U_{rep} &= -k_r (1/\rho - 1/\rho_0) (1/\rho^2) \nabla \rho \end{aligned}$$

Il gradiente $\nabla \rho$ è il versore calcolato da `obstacle_distances`. Il potenziale cresce all'infinito avvicinandosi all'ostacolo, che quindi non viene mai attraversato dalla discesa.

Generazione della direzione di moto

Il percorso è ottenuto con una discesa del gradiente a passo normalizzato: $p(k+1) = p(k) - \eta \nabla U / \|\nabla U\|$, con $\eta = p.apf.step$ (ridotto vicino al goal). La normalizzazione rende uniforme la spaziatura dei punti; se il passo porta in collisione, viene dimezzato.

Problema dei minimi locali

Il limite principale del metodo è che U può avere **minimi locali** diversi dal goal, dove $\nabla U = 0$: la forza attrattiva e quelle repulsive si bilanciano e la discesa si blocca. I casi tipici sono:

- **ostacolo concavo** (a U) aperto verso il robot: la forza attrattiva spinge il robot dentro la "tasca" e le pareti lo trattengono. Lo scenario `trap` riproduce esattamente questa situazione;

- **ostacolo allineato** tra robot e goal: sulla retta robot–goal, dietro un ostacolo simmetrico, le forze si annullano (punto di sella o minimo);
- **passaggi stretti** tra due ostacoli vicini: le repulsioni sommate chiudono il corridoio anche se geometricamente è percorribile;
- **goal vicino a un ostacolo** (problema *GNRON*, Goal Non-Reachable with Obstacles Nearby): la repulsione è ancora attiva sul goal, che quindi non è più il minimo del potenziale totale. Nello scenario *base* il goal è a 0,8 m dai bordi e senza correzione il robot si ferma prima.

Il progetto **rileva** il minimo locale quando lo spostamento negli ultimi N passi è minore di una soglia (o il gradiente è quasi nullo) e adotta tre contromisure, attivabili con i parametri:

1. **Correzione GNRON (Ge e Cui)**. Entro un raggio R_g dal goal il potenziale repulsivo è moltiplicato per $f = (d/R_g)^n$; il gradiente diventa $f \nabla U_{rep} + U_{rep} \nabla f$ con $\nabla f = n d^{n-2} (p - p_g)/R_g^n$. Così il goal resta il minimo globale (*goal_rep_n*, *goal_rep_radius*).
2. **Random walk di fuga** (strategia di Barraquand–Latombe): dal minimo si esegue una passeggiata casuale collision-free la cui lunghezza raddoppia a ogni nuova fuga, poi si riprende la discesa.
3. **Ostacoli virtuali**: il minimo rilevato diventa una sorgente repulsiva fittizia che “riempie” il pozzo, così la discesa successiva non vi ricade.

Con *p.apf.escape = false* il pianificatore si ferma e segnala la posizione del minimo: è il modo per mostrare il problema. Il metodo resta comunque **non completo**: le contromisure aumentano molto la probabilità di successo, ma il percorso ottenuto può essere lungo e irregolare (vedi Capitolo 8). Per ambienti complessi sono preferibili i metodi su grafo o i potenziali privi di minimi locali (funzioni di navigazione, potenziali armonici).

3.6 Controllo cinematico del robot

Dato un punto obiettivo p_t (il waypoint corrente oppure il riferimento continuo), si definiscono la distanza ρ e l'errore angolare α tra l'asse del robot e la direzione del target:

$$\rho = \|p_t - p\| \quad \alpha = \text{atan2}(y_t - y, x_t - x) - \theta \in (-\pi, \pi]$$

La legge di controllo richiesta dalla traccia è:

$$v = k_p \rho \quad \omega = k_\alpha \alpha$$

Analisi di convergenza

In coordinate polari, per un target fisso, $\dot{\rho} = -v \cos \alpha$ e $\dot{\alpha} = -\omega + (v/\rho) \sin \alpha$. Sostituendo la legge di controllo:

$$\dot{\rho} = -k_p \rho \cos \alpha \quad \dot{\alpha} = -k_\alpha \alpha + k_p \sin \alpha \cos \alpha$$

Linearizzando attorno a $(\rho, \alpha) = (0, 0)$ si ottiene $\dot{\alpha} \approx -(k_\alpha - k_p) \alpha$ e $\dot{\rho} \approx -k_p \rho$: l'equilibrio è asintoticamente stabile se $k_p > 0$ e $k_\alpha > k_p$.

Il codice segnala con un warning l'uso di guadagni che non rispettano la condizione $k_\alpha > k_p$. Il Capitolo 8 mostra numericamente che con k_α troppo vicino a k_p l'inseguimento peggiora fino al contatto con un ostacolo.

Come variante opzionale (*p.ctrl.cos_alpha_scaling = true*, default) si usa $v = k_p \rho \cdot \max(\cos \alpha, 0)$: il robot rallenta mentre si orienta e non avanza se il target è alle sue spalle, evitando ampie traiettorie ad arco vicino agli ostacoli. Ponendo il parametro a false si ottiene esattamente la legge della traccia.

Conversione da percorso a riferimento

Come richiesto dal punto 5.1 della traccia sono disponibili due modalità (`p.ctrl.ref_mode`):

- **waypoints**: il target è il waypoint corrente; quando il robot entra nel cerchio di raggio `switch_radius` si passa al successivo. L'ultimo waypoint è q_f .
- **continuous**: il target è il punto $p_{ref}(s)$ del percorso all'ascissa curvilinea s , che avanza con velocità costante v_{ref} (riferimento continuo nel tempo). Se il robot accumula un ritardo maggiore di `max_lag`, il riferimento si ferma ad aspettarlo; a regime l'inseguimento ha un ritardo $p \approx v_{ref}/k_p$.

Allineamento finale e limiti

Raggiunta q_f entro `goal_tol`, se `align_final_heading = true` il robot ruota sul posto ($v = 0$, $\omega = k_\theta \cdot (\theta_f - \theta)$) fino a raggiungere l'orientamento finale entro `heading_tol`. In ogni istante v e ω sono limitati a v_{max} e ω_{max} , convertiti in (ω_R, ω_L) e saturati preservando la curvatura. È possibile aggiungere rumore gaussiano alle ruote (`wheel_noise_std`) per valutare la robustezza.

3.7 Indicatori di prestazione

- **Errore di tracking laterale** $e_{ct}(t)$: distanza minima della posizione del robot dalla polilinea pianificata (cross-track error). Se ne riportano valore RMS e massimo.
- **Distanza dal target** $p(t)$: misura dell'avanzamento lungo il percorso.
- **Clearance minima**: distanza minima tra il *bordo* del robot e gli ostacoli, sul percorso pianificato e su quello eseguito; un valore ≤ 0 indica collisione.
- Errori finali di posizione e orientamento, tempo di pianificazione ed esecuzione, lunghezze dei percorsi, massima $|\omega|$ delle ruote e frazione di tempo in saturazione.

4. Descrizione passo dopo passo dell'implementazione

Passo 1 – Definizione dei parametri

`default_params.m` crea la struttura `p` divisa in sottostrutture: `p.robot`, `p.env`, `p.cell`, `p.prm`, `p.rrt`, `p.apf`, `p.post`, `p.ctrl`, `p.sim`, `p.viz`. L'utente modifica solo i campi che vuole cambiare (in `main.m` o da Command Window); tutti gli altri mantengono il default. L'elenco completo è nel Capitolo 5.

Passo 2 – Costruzione e verifica dell'ambiente

`build_environment(p)` legge lo scenario (`base`, `narrow`, `trap` oppure `custom`), crea gli ostacoli con `make_obstacle`, calcola il raggio gonfiato $R + m$ e determina q_0 e q_f (quelle dello scenario oppure quelle fornite dall'utente). `validate_params` verifica segni e domini dei parametri, i nomi delle opzioni e che q_0 e q_f non siano in collisione, segnalando l'errore con un messaggio esplicito.

Passo 3 – Pianificazione

`plan_path` imposta il seme casuale (per avere risultati riproducibili), lancia il pianificatore scelto e misura il tempo. Tutti i pianificatori hanno la stessa interfaccia `[path, info] = plan_xxx(env, q0, qf, parametri)` e restituiscono un percorso che parte esattamente da q_0 e termina esattamente in q_f , più una struttura `info` con la diagnostica (griglia, roadmap, albero, minimi locali...). In caso di fallimento `info.message` spiega il motivo e `run_project` termina con un warning che invita a modificare i parametri.

Passo 4 – Post-elaborazione e waypoint

Se `p.post.shortcut` è attivo il percorso viene semplificato con lo string pulling; poi `resample_path` genera i waypoint con spaziatura massima `waypoint_spacing`. Il risultato è la sequenza (x_i, y_i) passata al controllore.

Passo 5 – Simulazione in anello chiuso

`simulate_robot` esegue, per ogni passo dt fino a t_{max} :

1. scelta del target (waypoint corrente con commutazione, oppure punto del riferimento continuo);
2. calcolo di (v, ω) con `controller_rho_alpha` (o della rotazione finale nella fase di allineamento);
3. limitazione di v e ω , conversione in (ω_R, ω_L) con `unicycle_to_wheels`, saturazione e rumore opzionale;
4. ricostruzione di (v, ω) effettivi dalle ruote e integrazione dello stato con `integrate_step`, che usa `diffdrive_dynamics`, cioè il modello differenziale;
5. registrazione di stato, ingressi, p , α , target e fase (1 = inseguimento, 2 = allineamento, 3 = arrivato).

Passo 6 – Analisi

`compute_metrics` calcola gli indicatori del paragrafo 3.7; `print_metrics` li stampa in Command Window; `compare_planners` ripete l'esperimento per più metodi (e più semi, per i metodi stocastici) e produce una tabella di confronto.

Passo 7 – Visualizzazione

- **Figura del pianificatore** (`plot_planner_debug`): celle libere/occupate e nodi ritratti, roadmap PRM, albero RRT, oppure curve di livello del potenziale APF con il campo $-\nabla U$ e i minimi locali rilevati.
- **Traiettorie**: percorso pianificato, waypoint e traiettoria eseguita sopra l'ambiente con q_0 e q_f orientate.
- **Errore di tracking**: $e_{ct}(t)$ e $p(t)$.
- **Velocità delle ruote**: $\omega_R(t)$, $\omega_L(t)$ con i limiti di saturazione, e $v(t)$, $\omega(t)$.
- **Stato**: $x(t)$, $y(t)$, $\theta(t)$ (θ srotolata).
- **Animazione**: corpo del robot, due ruote, prua, scia della traiettoria e target corrente; con `p.viz.save_gif = true` viene salvata come GIF in `p.viz.output_dir`.

5. Parametri configurabili

Nessun valore numerico è scritto nelle funzioni: tutti si trovano in `config/default_params.m`, che restituisce una struttura `p` organizzata in sotto-strutture. Le tabelle seguenti sono generate direttamente da quel file e riportano, per ogni campo, il valore di default e il significato. Per cambiare un valore lo si sovrascrive in `main.m` (sezione PARAMETRI UTENTE) oppure da Command Window dopo `p = default_params();`.

Unità di misura del Sistema Internazionale in tutto il progetto: m, s, rad, m/s, rad/s. Per gli angoli si può usare `pi/2` oppure `deg2rad(...)`.

5.1 Robot (p.robot)

Campo	Default	Significato
<code>p.robot.r</code>	0.05	raggio delle ruote <code>r</code> [m]
<code>p.robot.d</code>	0.30	distanza tra le ruote (carreggiata) <code>d</code> [m]
<code>p.robot.radius</code>	0.20	raggio del cerchio d'ingombro del robot [m]
<code>p.robot.wheel_max</code>	20	$ \omega_R , \omega_L $ massime [rad/s]
<code>p.robot.v_max</code>	0.8	velocità lineare massima [m/s]
<code>p.robot.w_max</code>	2.5	velocità angolare massima [rad/s]

5.2 Ambiente (p.env)

Campo	Default	Significato
<code>p.env.scenario</code>	'base'	'base' 'narrow' 'trap' 'custom'
<code>p.env.bounds</code>	[]	[xmin xmax ymin ymax]; [] = quelli dello scenario
<code>p.env.custom_obstacles</code>	{}	ostacoli se scenario = 'custom' (vedi <code>scenario_library</code>)
<code>p.env.safety_margin</code>	0.05	marginale aggiunto al raggio del robot [m]

5.3 Configurazioni iniziale e finale (p.q0, p.qf)

Campo	Default	Significato
<code>p.q0</code>	[]	[x0 y0 theta0]; [] = configurazione iniziale dello scenario
<code>p.qf</code>	[]	[xf yf thetalf]; [] = configurazione finale dello scenario

5.4 Pianificazione

Campo	Default	Significato
<code>p.planner</code>	'prm'	'cell' 'prm' 'rrt' 'apf'
<code>p.seed</code>	1	seme casuale (riproducibilità di PRM/RRT/APF)

5.5 Cell decomposition (p.cell)

Campo	Default	Significato
p.cell.resolution	0.20	lato della cella [m]
p.cell.connectivity	8	4 oppure 8 vicini
p.cell.conservative	true	true: cella libera solo se lo è tutta
p.cell.clearance_weight	0.30	peso [m] che allontana il cammino dagli ostacoli (0 = più corto)
p.cell.search	'dijkstra'	'dijkstra' 'astar'

5.6 PRM (p.prm)

Campo	Default	Significato
p.prm.n_samples	400	campioni per tentativo
p.prm.k_neighbors	10	vicini massimi tentati per nodo
p.prm.connection_radius	2.5	raggio massimo di connessione [m]
p.prm.max_attempts	4	tentativi se il grafo non connette q_0 - q_f
p.prm.growth	1.5	fattore di crescita dei campioni per tentativo
p.prm.search	'dijkstra'	'dijkstra' 'astar'

5.7 RRT (p.rrt)

Campo	Default	Significato
p.rrt.max_iter	6000	iterazioni massime
p.rrt.step	0.30	passo di espansione [m]
p.rrt.goal_bias	0.10	probabilità di campionare il goal
p.rrt.goal_tol	0.30	distanza per tentare la connessione al goal [m]

5.8 Artificial Potential Fields (p.apf)

Campo	Default	Significato
p.apf.k_att	1.0	guadagno attrattivo
p.apf.d_star	2.0	soglia quadratico/conico del potenziale attrattivo [m]
p.apf.k_rep	1.0	guadagno repulsivo
p.apf.rho0	0.8	raggio d'influenza degli ostacoli [m]
p.apf.goal_rep_n	2	correzione GNRON: esponente n (0 = Khatib classico)
p.apf.goal_rep_radius	1.0	entro questa distanza dal goal la repulsione è scalata $(d/R)^n$ [m]
p.apf.step	0.05	passo della discesa del gradiente [m]
p.apf.max_iter	4000	passi massimi

Campo	Default	Significato
p.apf.goal_tol	0.08	tolleranza di arrivo [m]
p.apf.stall_window	40	finestra (passi) per rilevare lo stallo
p.apf.stall_dist	0.10	spostamento minimo nella finestra [m]
p.apf.escape	true	true: fuga dai minimi locali con random walk
p.apf.escape_steps	40	passi del primo random walk (raddoppiano a ogni fuga)
p.apf.escape_growth_max	16	fattore massimo di crescita del random walk
p.apf.escape_step	0.10	lunghezza di un passo di random walk [m]
p.apf.max_escapes	25	tentativi massimi di fuga
p.apf.k_virtual	0.5	guadagno degli ostacoli virtuali posti sui minimi (0 = disattivi)
p.apf.rho_virtual	1.0	raggio d'influenza degli ostacoli virtuali [m]

5.9 Post-elaborazione del percorso (p.post)

Campo	Default	Significato
p.post.shortcut	true	semplificazione "string pulling" collision-free
p.post.waypoint_spacing	0.25	distanza tra waypoint dopo il ricampionamento [m]
p.post.collision_step	0.02	passo di campionamento nel test dei segmenti [m]

5.10 Controllo (p.ctrl)

Campo	Default	Significato
p.ctrl.k_rho	1.0	$v = k_rho * rho$
p.ctrl.k_alpha	3.0	$\omega = k_alpha * \alpha$
p.ctrl.ref_mode	'waypoints'	'waypoints' 'continuous'
p.ctrl.switch_radius	0.25	[waypoints] raggio di commutazione [m]
p.ctrl.v_ref	0.40	[continuous] velocità del riferimento [m/s]
p.ctrl.max_lag	0.50	[continuous] ritardo max prima di fermare il riferimento [m]
p.ctrl.cos_alpha_scaling	true	v moltiplicata per $\max(\cos(\alpha), 0)$
p.ctrl.goal_tol	0.05	tolleranza di posizione finale [m]
p.ctrl.align_final_heading	true	ruota sul posto fino a θ_f
p.ctrl.k_theta	2.0	guadagno dell'allineamento finale
p.ctrl.heading_tol	$2\pi/180$	tolleranza di orientamento finale [rad]

5.11 Simulazione (p.sim)

Campo	Default	Significato
p.sim.dt	0.02	passo di integrazione [s]
p.sim.t_max	150	durata massima [s]
p.sim.integrator	'rk4'	'euler' 'rk4' 'exact'
p.sim.wheel_noise_std	0.0	rumore gaussiano sulle ruote [rad/s] (0 = ideale)

5.12 Visualizzazione (p.viz)

Campo	Default	Significato
p.viz.show_plots	true	grafici dei risultati
p.viz.show_planner_debug	true	grafico della struttura del pianificatore
p.viz.animate	true	animazione del moto
p.viz.frame_skip	5	disegna 1 frame ogni N passi
p.viz.pause	0.001	pausa tra frame [s]
p.viz.save_gif	false	salva l'animazione come GIF
p.viz.save_figures	false	salva i grafici come PNG
p.viz.output_dir	'output'	
p.viz.gif_file	'robot_motion.gif'	
p.verbose	true	stampa a console

5.13 Definire uno scenario personalizzato

Con `p.env.scenario = 'custom'` si definiscono liberamente bordi, ostacoli e configurazioni:

```
p = default_params();
p.env.scenario = 'custom';
p.env.bounds = [0 8 0 6]; % [xmin xmax ymin ymax]
p.env.custom_obstacles = { ...
    make_obstacle('circle', [3 3], 0.8), ... % centro, raggio
    make_obstacle('polygon', [5 1; 6 1; 6 4; 5 4]), ... % vertici
    make_obstacle('polygon', [1 4.5; 2.5 4; 2 5.5]) };
p.q0 = [0.7 0.7 0]; % [x y theta]
p.qf = [7.3 5.3 pi/2];
out = run_project(p);
```

5.14 Indicazioni per la scelta dei valori

- **Risoluzione delle celle:** deve essere minore di circa metà della larghezza dei passaggi liberi; valori piccoli aumentano il tempo di calcolo con legge quadratica.
- **PRM:** aumentare `n_samples` o `connection_radius` se q_0 e q_f restano disconnessi (passaggi stretti); `k_neighbors` 8–15 è un buon compromesso.
- **RRT:** `step` paragonabile alla larghezza dei passaggi; `goal_bias` 0,05–0,2.
- **APF:** `rho0` maggiore del raggio del robot ma minore della distanza tra ostacoli vicini; `k_rep` troppo grande chiude i corridoi, troppo piccolo fa avvicinare il robot agli ostacoli.

- **Controllore:** rispettare $k_a > k_p$ (tipicamente $k_a \approx 3k_p$). Guadagni più alti riducono i tempi ma aumentano le velocità delle ruote fino alla saturazione.
- **Simulazione:** con RK4, $dt = 0,01\text{--}0,05$ s è adeguato; con Eulero conviene un dt più piccolo.

6. Installazione, esecuzione e test

6.1 Requisiti

- MATLAB R2016b o successivo (per le funzioni con espansione implicita); **nessun toolbox aggiuntivo**.
- In alternativa GNU Octave ≥ 6 (verificato con Octave 8.4).

6.2 Avvio rapido

1. Estrarre **RobotMobileDifferenziale.zip** in una cartella a scelta (l'archivio è anche incorporato nella copertina di questo documento: doppio clic sull'icona per aprirlo).
2. Aprire MATLAB e impostare come Current Folder la cartella **RobotMobileDifferenziale**.
3. Aprire **main.m**, modificare se necessario la sezione **PARAMETRI UTENTE**.
4. Eseguire **main** (pulsante Run oppure da Command Window). **main.m** aggiunge automaticamente al path tutte le sottocartelle.

Vengono stampati i risultati in Command Window, aperte le figure (pianificatore, traiettorie, errore, velocità delle ruote, stato) e avviata l'animazione. L'output tipico è il seguente (scenario base, PRM, valori di default):

```
[PRM] pianificazione: ok (x.xxx s)

----- RISULTATI (PRM) -----
Tempo di pianificazione      :    x.xxx s
Lunghezza percorso pianificato:  12.986 m
Lunghezza percorso eseguito  :  12.9xx m
Goal raggiunto               :      SI
Tempo di esecuzione         :   38.46 s
Errore finale di posizione   :  0.04xx m
Errore finale di orientamento:  1.9xx deg
Errore di tracking RMS / max :  0.0089 / 0.0583 m
Distanza min. dagli ostacoli :  0.0xx m (pianificato) / 0.057 m (eseguito)
Collisioni                   :      NO
|omega ruota| massima        :  11.90 rad/s (saturazione 0.0% del tempo)
-----
```

I tempi di pianificazione dipendono dalla macchina; in MATLAB sono in genere inferiori a quelli misurati in GNU Octave e riportati nel Capitolo 8.

6.3 Uso da Command Window

```
addpath(genpath(pwd));           % una sola volta per sessione
p = default_params();

% esempio 1: RRT nello scenario con passaggio stretto
p.planner = 'rrt'; p.env.scenario = 'narrow';
out = run_project(p);

% esempio 2: APF nello scenario trappola, prima senza e poi con la fuga
p = default_params(); p.planner = 'apf'; p.env.scenario = 'trap';
p.apf.escape = false; out1 = run_project(p); % si blocca nel minimo locale
p.apf.escape = true;  out2 = run_project(p); % esce e raggiunge il goal

% esempio 3: riferimento continuo, guadagni diversi, GIF salvata
p = default_params();
p.ctrl.ref_mode = 'continuous'; p.ctrl.v_ref = 0.3;
p.ctrl.k_rho = 1.5; p.ctrl.k_alpha = 4.5;
p.viz.save_gif = true; p.viz.save_figures = true; % in ./output
out = run_project(p);

% esempio 4: solo calcolo, senza grafica (utile per esperimenti in batch)
p.viz.show_plots = false; p.viz.animate = false; p.viz.show_planner_debug = false;
```

```
out = run_project(p); disp(out.metrics.ct_rms)
```

La struttura restituita `out` contiene: `success`, `env`, `q0`, `qf`, `path` (percorso pianificato), `waypoints`, `plan_info` (diagnostica del pianificatore), `sim` ($t, q, \omega_R, \omega_L, v, \omega, \rho, \alpha$, target, fase) e `metrics`.

6.4 Test automatici

Per verificare l'intero progetto:

```
addpath(genpath(pwd));
ok = run_all_tests(); % true se tutti i test sono superati
```

La suite comprende 18 verifiche:

Gruppo	Verifica
Modello	invertibilità $(v, \omega) \leftrightarrow (\omega_R, \omega_L)$; moto rettilineo $x = r\omega t$ con ruote uguali; rotazione pura $\theta = 2r\omega t/d$ con ruote opposte; RK4 coincidente con la soluzione esatta ($< 1e-8$); saturazione che rispetta il limite e preserva la curvatura
Geometria	distanza da un cerchio; distanza negativa interna e positiva esterna da un poligono; segmento che attraversa un ostacolo rifiutato; segmento libero accettato
Pianificatori	Cell, PRM, RRT e APF producono un percorso da q_0 a q_f collision-free (anche dopo shortcut e ricampionamento)
Minimi locali	APF senza fuga si blocca nella U dello scenario trap; con la fuga raggiunge il goal
Controllo	in modalità waypoints e continuous il robot raggiunge q_f entro le tolleranze di posizione e orientamento, senza collisioni e senza superare ω_{max}

Esito dell'esecuzione di riferimento: **18 / 18 test superati**.

6.5 Confronto dei pianificatori

```
p = default_params();
p.env.scenario = 'base';
T = compare_planners(p, {'cell', 'prm', 'rrt', 'apf'}, 5); % 5 semi per metodo
```

La funzione stampa una tabella con percentuale di successo, tempo di pianificazione, lunghezza, tempo di esecuzione, errore di tracking RMS e massimo, clearance minima e massima velocità delle ruote (medie sulle esecuzioni riuscite).

6.6 Problemi comuni

Messaggio / sintomo	Causa e soluzione
" $q_0 \dots$ è in collisione o troppo vicina a un ostacolo/bordo"	q_0 o q_f distano meno di $R + m$ da un ostacolo o dal bordo: spostarle o ridurre <code>safety_margin</code> .
Cell: "nessun cammino nel grafo di adiacenza"	Passaggi più stretti di una cella: ridurre <code>cell.resolution</code> oppure porre <code>cell.conservative = false</code> .
PRM: " q_0 e q_f in componenti non connesse"	Aumentare <code>prm.n_samples</code> , <code>prm.connection_radius</code> o <code>prm.max_attempts</code> .

Messaggio / sintomo	Causa e soluzione
RRT: “goal non raggiunto entro max_iter”	Aumentare <code>rrt.max_iter</code> o <code>rrt.goal_bias</code> ; adattare <code>rrt.step</code> .
APF: “bloccato in un minimo locale”	Attivare <code>apf.escape</code> , aumentare <code>apf.max_escapes</code> ; ridurre <code>k_rep/rho0</code> se i corridoi sono chiusi.
“Goal non raggiunto entro t_max”	Aumentare <code>sim.t_max</code> o i guadagni del controllore.
Warning “k_alpha > k_rho”	Guadagni fuori dalla regione di convergenza: aumentare <code>ctrl.k_alpha</code> .
Animazione lenta	Aumentare <code>viz.frame_skip</code> o disattivare <code>viz.animate</code> .

7. Risultati delle simulazioni

Le figure seguenti sono state ottenute con i parametri di default (seme 1). I dati provengono dall'esecuzione del codice del progetto; il tracciamento delle figure per questo documento riproduce i grafici di `plot_results` e `plot_planner_debug`.

7.1 Traiettoria pianificata ed eseguita (PRM, scenario base)

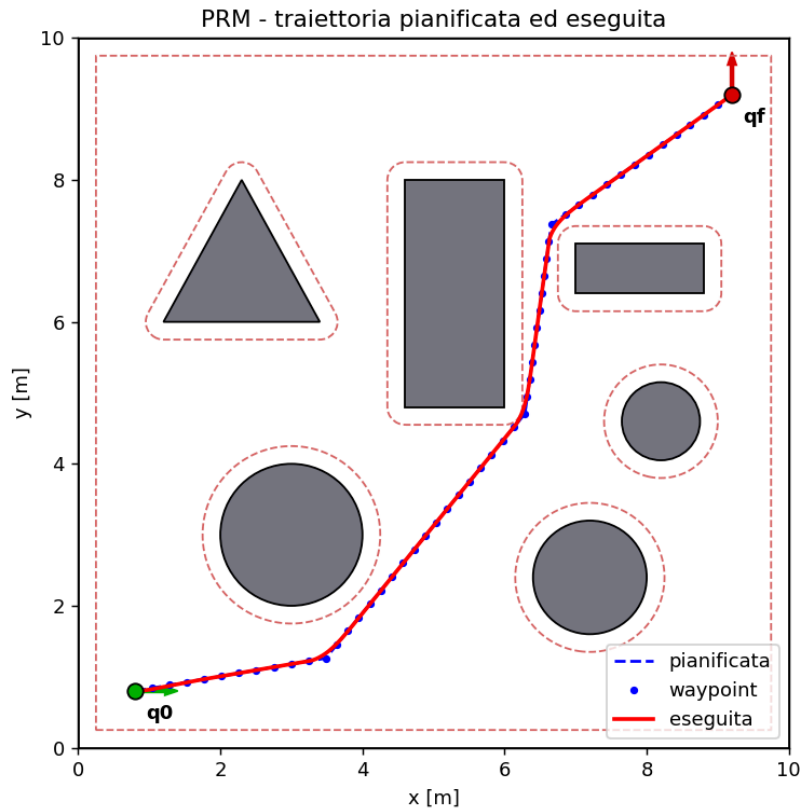


Figura 1 – Ostacoli (grigio), contorno degli ostacoli gonfiati (rosso tratteggiato), percorso pianificato, waypoint e traiettoria eseguita.

La traiettoria eseguita (rossa) è praticamente sovrapposta a quella pianificata. Il robot parte orientato lungo x , si allinea al primo segmento e raggiunge q_f con l'orientamento richiesto $\theta_f = \pi/2$. Gli scostamenti maggiori si hanno in corrispondenza dei vertici, dove il robot, non potendo ruotare istantaneamente mentre avanza, raccorda gli spigoli con un piccolo arco.

7.2 Errore di tracking

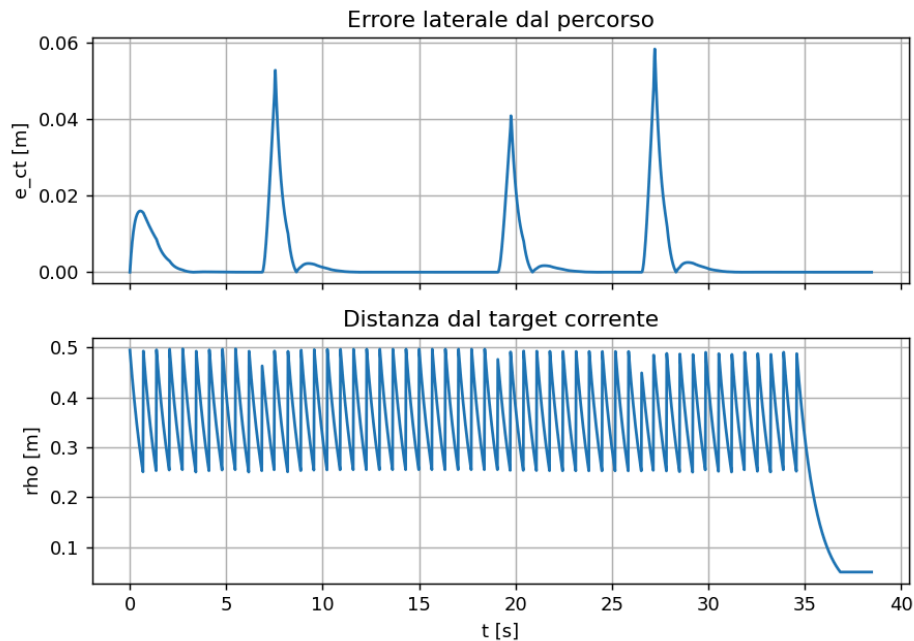


Figura 2 – Errore laterale dal percorso e distanza dal target corrente.

L'errore laterale resta sotto 6 cm con valore RMS di circa 9 mm. I picchi coincidono con i cambi di direzione del percorso; sui tratti rettilinei l'errore converge a zero, come previsto dall'analisi di stabilità. La distanza ρ dal target ha l'andamento a dente di sega tipico della commutazione dei waypoint: cresce di un passo a ogni commutazione e decresce mentre il robot si avvicina.

7.3 Velocità delle ruote

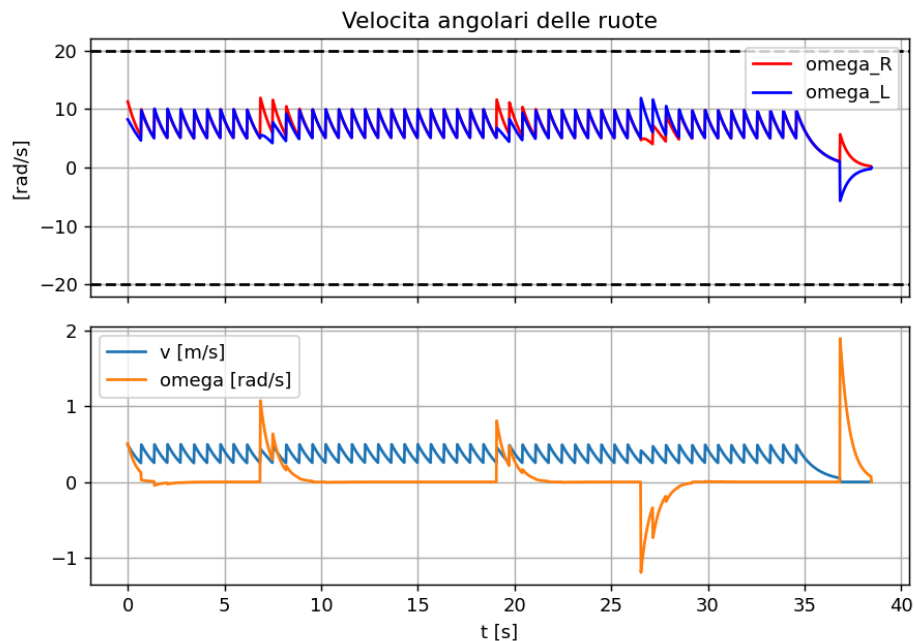


Figura 3 – Velocità angolari delle ruote con i limiti di saturazione; velocità lineare e angolare del veicolo.

Con $\omega_R = \omega_L$ il robot avanza in linea retta; i picchi di segno opposto corrispondono alle correzioni di direzione ai vertici. La rotazione finale sul posto ($\omega_R = -\omega_L$, $v = 0$) è visibile alla fine. Le velocità restano entro ± 20 rad/s senza saturazione con i guadagni di default.

7.4 Struttura dei pianificatori

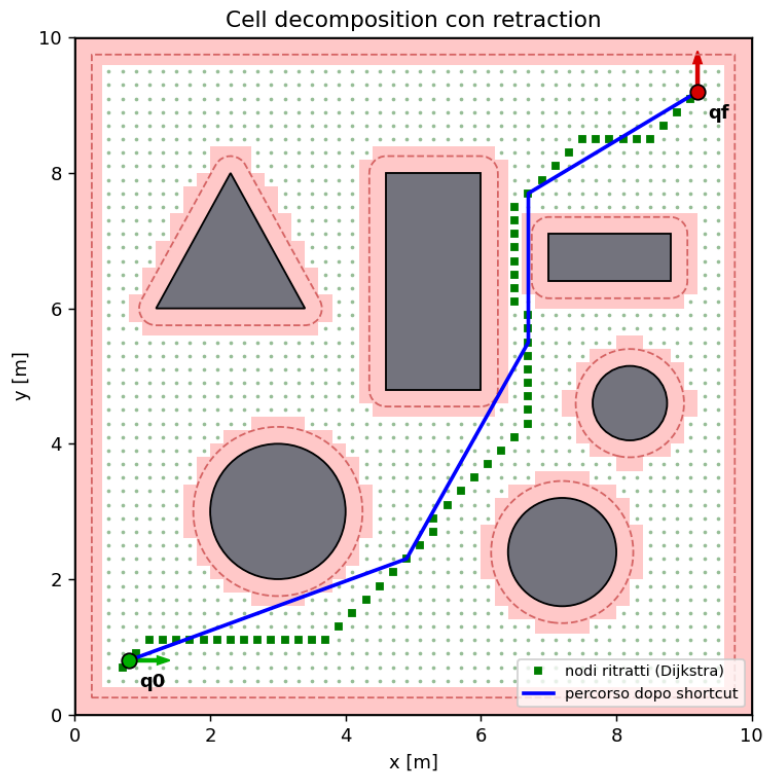


Figura 4 – Cell decomposition: celle occupate (rosa), centri delle celle libere, nodi ritratti del cammino di Dijkstra e percorso dopo lo shortcut.

Grazie al costo che penalizza la bassa clearance, i nodi ritratti si mantengono al centro dei corridoi (si veda il tratto verticale tra il rettangolo e il cerchio di destra). Lo shortcut elimina poi la “scalettatura” della griglia.

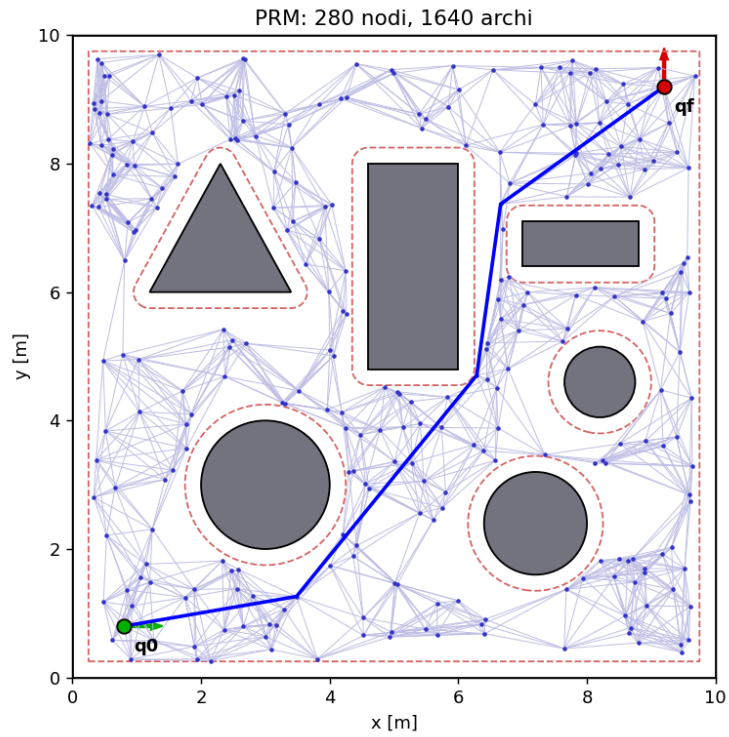


Figura 5 – Roadmap PRM: nodi campionati, archi collision-free e cammino minimo.

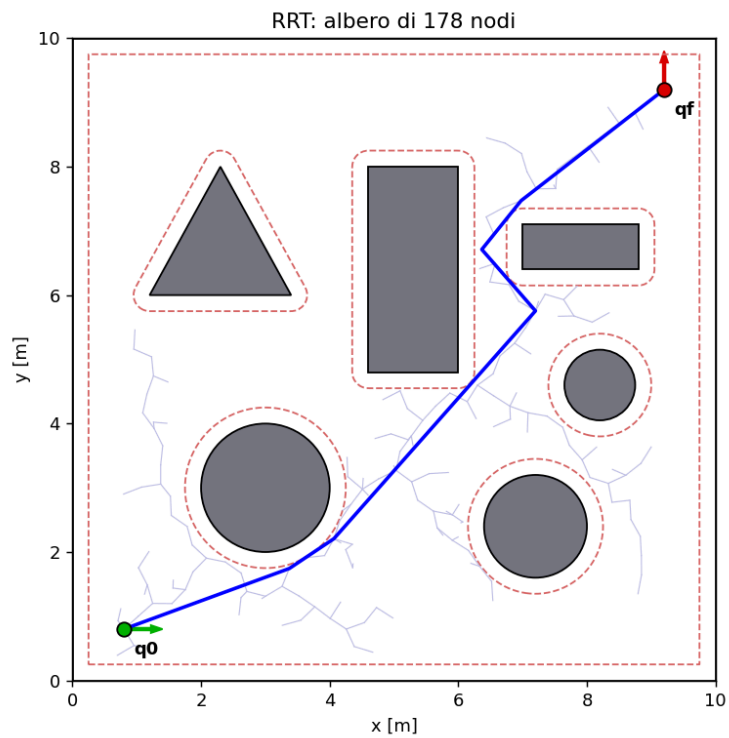


Figura 6 – Albero RRT e percorso estratto.

7.5 Campi di potenziale e minimi locali

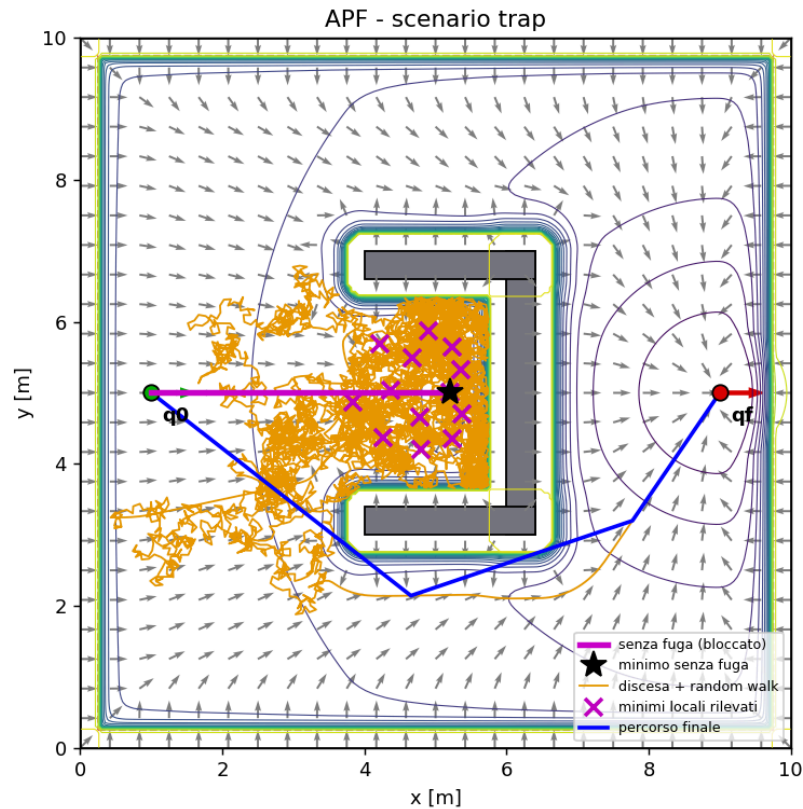


Figura 7 – Scenario trap: curve di livello del potenziale e campo $-\nabla U$. Senza fuga la discesa si arresta nel minimo locale dentro la U (stella); con random walk e ostacoli virtuali il pianificatore esce e raggiunge il goal.

Il campo $-\nabla U$ mostra chiaramente il bacino di attrazione della U: tutte le frecce nella zona antistante convergono verso la tasca. La traiettoria senza fuga (viola) termina nel punto (5,2; 5,0), dove le repulsioni delle tre pareti bilanciano l'attrazione del goal. Con la fuga sono stati necessari 13 tentativi: i primi random walk, brevi, restano nel bacino (crochette), mentre quelli successivi, più lunghi e aiutati dagli ostacoli virtuali, portano il robot fuori dalla U, da dove la discesa aggira l'ostacolo.

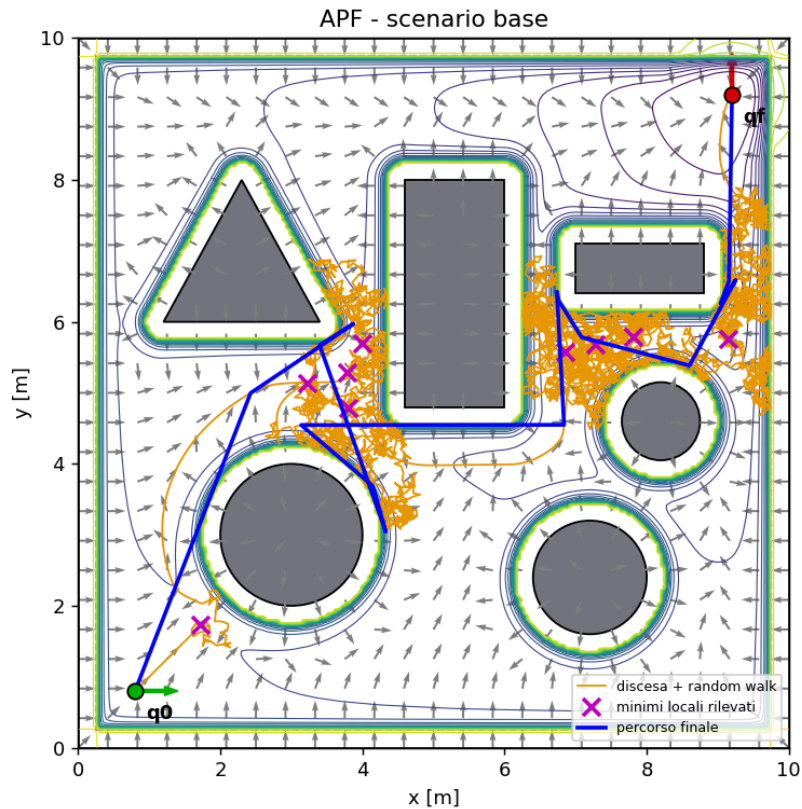


Figura 8 – Scenario base con APF: minimi locali davanti al cerchio allineato con il goal e tra il rettangolo e gli ostacoli di destra.

Anche nello scenario base compaiono minimi locali: il primo davanti al cerchio posto esattamente sulla retta q_0 – q_f , gli altri nelle strettoie dove le repulsioni di ostacoli vicini si sommano. Senza la correzione GNRON il robot si fermerebbe inoltre a circa 0,5 m dal goal, a causa della repulsione dei bordi.

8. Analisi dei risultati

8.1 Confronto dei pianificatori

Risultati di `compare_planners` (parametri di default, seme 1). I tempi sono misurati in GNU Octave e sono quindi da leggere in senso relativo.

Scenario	Metodo	t plan. [s]	L [m]	t goal [s]	e RMS [m]	e max [m]	clearance [m]	$ \omega $ max [rad/s]
base	CELL	0,59	13,15	39,2	0,0104	0,071	0,067	12,3
base	PRM	2,95	12,99	38,5	0,0089	0,058	0,057	11,9
base	RRT	0,54	13,37	39,6	0,0130	0,089	0,051	12,8
base	APF	15,70	24,19	64,8	0,0195	0,093	0,026	13,6
narrow	CELL	0,54	11,35	33,6	0,0133	0,086	0,053	13,6
narrow	PRM	3,27	10,97	32,3	0,0142	0,062	0,053	13,8
narrow	RRT	0,73	11,07	32,1	0,0140	0,070	0,089	13,8
trap	CELL	0,68	10,31	31,2	0,0137	0,077	0,101	13,8
trap	PRM	4,33	10,07	30,7	0,0137	0,065	0,137	14,0
trap	RRT	0,49	10,20	31,0	0,0137	0,069	0,058	13,7

Qualità del percorso. Dopo lo shortcut, PRM produce i percorsi più corti in tutti gli scenari, perché la roadmap copre bene lo spazio e Dijkstra trova il minimo su di essa. La griglia è vicina all'ottimo ma, a causa del peso di clearance, privilegia la distanza dagli ostacoli (clearance maggiore nello scenario base). RRT dà percorsi leggermente più lunghi: l'albero cresce verso il goal ma non ottimizza la lunghezza (RRT* lo farebbe).

Tempo di calcolo. RRT e la griglia sono i più rapidi su questi scenari; PRM costa di più perché verifica molti archi, ma la roadmap è riutilizzabile per altre interrogazioni. APF è il più lento perché ogni passo richiede il calcolo del gradiente e, soprattutto, perché deve gestire i minimi locali: il percorso finale (24 m contro 13 m) conserva parte dei random walk, e il robot impiega quasi il doppio del tempo.

Sicurezza. Tutti i percorsi sono collision-free e la clearance eseguita resta positiva. I valori di pochi centimetri sono attesi: i percorsi semplificati passano a distanza pari al margine di sicurezza (5 cm) dagli ostacoli gonfiati. Per avere più margine si aumenta `safety_margin` o `cell.clearance_weight`.

8.2 Effetto dei guadagni del controllore

Scenario base, RRT, modalità waypoints; si variano solo i guadagni:

k_p	k_a	Esito	t goal [s]	e RMS [m]	e max [m]	$ \omega $ max [rad/s]	Saturazione
0,5	1,5	ok	77,6	0,0133	0,089	6,4	0%
1,0	3,0	ok	39,6	0,0130	0,089	12,8	0%
2,0	6,0	ok	21,3	0,0114	0,079	20,0	2,0%

k_p	k_α	Esito	t goal [s]	e RMS [m]	e max [m]	$ \omega $ max [rad/s]	Saturazione
1,0	1,2	contatto con un ostacolo	40,7	0,0277	0,091	10,6	0%

Il tempo di esecuzione è inversamente proporzionale a k_p , mentre l'errore di tracking dipende soprattutto dal rapporto k_α/k_p . Con guadagni alti le ruote raggiungono la saturazione: la saturazione che preserva la curvatura mantiene comunque il robot sul percorso, rallentandolo. Con $k_\alpha = 1,2 \approx k_p$, al limite della condizione di stabilità $k_\alpha > k_p$, l'orientamento converge troppo lentamente, l'errore RMS raddoppia e nei pressi di uno spigolo il robot tocca l'ostacolo (clearance $-1,7$ mm): è la verifica numerica della condizione derivata nel paragrafo 3.6.

8.3 Integrazione numerica, rumore e variante della legge di controllo

- Con $dt = 0,1$ s i tre integratori danno risultati praticamente coincidenti (stesso tempo di arrivo ed errore finale), perché in un passo il robot si sposta di pochi centimetri. RK4 coincide con la soluzione esatta a meno di $1e-8$ (test automatico); Eulero diverge dalla soluzione esatta solo con passi molto più grandi o velocità angolari elevate.
- Con rumore gaussiano di 2 rad/s sulle ruote (circa il 15% della velocità tipica) il robot raggiunge comunque il goal: l'errore RMS sale da 1,3 a 1,5 cm, l'anello chiuso compensa i disturbi.
- Disattivando la modulazione $\max(\cos \alpha, 0)$, cioè usando esattamente $v = k_p \rho$, il robot raggiunge ugualmente il goal senza collisioni con un errore simile (RMS 1,2 cm): con waypoint ravvicinati α resta piccolo e le due leggi quasi coincidono. La modulazione diventa importante quando il target è alle spalle del robot, per esempio alla partenza con orientamento opposto al percorso.
- La modalità a riferimento continuo produce un moto a velocità più uniforme (36 s contro 38,5 s, errore RMS 1,2 cm) perché il target scorre con continuità invece di saltare tra i waypoint.

8.4 Considerazioni conclusive

Il progetto soddisfa tutti i punti della traccia e li estende: quattro pianificatori intercambiabili, due modalità di riferimento, tre integratori, saturazione realistica degli attuatori, metriche quantitative, test automatici e parametrizzazione completa. Le simulazioni mostrano che il controllore $v = k_p \rho$, $\omega = k_\alpha \alpha$, unito a un percorso ben campionato, insegue il riferimento con errore dell'ordine del centimetro, purché sia rispettata la condizione $k_\alpha > k_p$. Tra i metodi di pianificazione, quelli basati su grafo (griglia, PRM, RRT) si sono rivelati affidabili e veloci, mentre APF, pur elegante e reattivo, richiede contromisure esplicite contro i minimi locali e produce percorsi meno efficienti.

Possibili estensioni: RRT o PRM per percorsi asintoticamente ottimi; smoothing con spline o curve di Dubins/Reeds-Shepp per percorsi a curvatura continua; controllore completo (ρ , α , β) per la stabilizzazione della posa; ostacoli dinamici e pianificazione reattiva; modello dinamico con limiti di accelerazione dei motori.

Appendice A – Codice sorgente completo

Di seguito è riportato integralmente il codice di tutti i file del progetto: prima i file della cartella principale, poi quelli delle sottocartelle nell'ordine della pipeline. Lo stesso codice è contenuto nell'archivio [RobotMobileDifferenziale.zip](#).

A.1 main.m

```
%% MAIN - Pianificazione e controllo di un robot mobile differenziale
%
% Punto d'ingresso del progetto. Uso:
% 1) aprire MATLAB nella cartella del progetto;
% 2) modificare la sezione "PARAMETRI UTENTE" qui sotto;
% 3) eseguire: >> main
%
% L'elenco completo dei parametri, con unita' di misura e significato, e'
% in config/default_params.m. Ogni campo non sovrascritto qui mantiene il
% valore di default.

clear; close all; clc;
root = fileparts(mfilename('fullpath'));
if isempty(root), root = pwd; end
addpath(genpath(root));

p = default_params();

%% ===== PARAMETRI UTENTE =====
% --- scenario e configurazioni
p.env.scenario = 'base';           % 'base' | 'narrow' | 'trap' | 'custom'
p.q0 = [];                         % es. [0.8 0.8 0]      ([ ] = default scenario)
p.qf = [];                         % es. [9.2 9.2 pi/2]

% --- robot
p.robot.r      = 0.05;             % raggio ruote [m]
p.robot.d      = 0.30;             % carreggiata [m]
p.robot.radius  = 0.20;             % ingombro [m]
p.robot.wheel_max = 20;            % [rad/s]

% --- pianificatore
p.planner = 'prm';                 % 'cell' | 'prm' | 'rrt' | 'apf'
p.seed    = 1;

% --- controllore
p.ctrl.k_rho  = 1.0;
p.ctrl.k_alpha = 3.0;
p.ctrl.ref_mode = 'waypoints'; % 'waypoints' | 'continuous'

% --- simulazione / grafica
p.sim.dt      = 0.02;
p.sim.integrator = 'rk4';          % 'euler' | 'rk4' | 'exact'
p.viz.animate  = true;
p.viz.save_gif  = false;

% --- esempio di scenario personalizzato (decommentare per usarlo)
% p.env.scenario = 'custom';
% p.env.bounds   = [0 8 0 6];
% p.env.custom_obstacles = { make_obstacle('circle', [3 3], 0.8), ...
%                             make_obstacle('polygon', [5 1; 6 1; 6 4; 5 4]) };
% p.q0 = [0.7 0.7 0]; p.qf = [7.3 5.3 pi/2];
%% =====

results = run_project(p);

%% (opzionale) confronto quantitativo fra i quattro pianificatori
% T = compare_planners(p, {'cell','prm','rrt','apf'}, 5);
```

A.2 run_project.m

```
function out = run_project(p)
%RUN_PROJECT Esegue l'intera pipeline: ambiente -> pianificazione -> controllo
```

```
% -> simulazione -> analisi -> grafici/animazione.
%
% out = run_project(p) con p ottenuta da default_params() e personalizzata.
% out.success, out.env, out.q0, out.qf, out.path, out.waypoints,
% out.plan_info, out.sim, out.metrics
if nargin < 1, p = default_params(); end
[env, q0, qf] = build_environment(p);
validate_params(p, env, q0, qf);
out = struct('success', false, 'env', env, 'q0', q0, 'qf', qf, 'path', [], ...
            'waypoints', [], 'plan_info', [], 'sim', [], 'metrics', []);

% ---- 1) pianificazione
[path, waypoints, info] = plan_path(env, q0, qf, p);
out.plan_info = info; out.path = path; out.waypoints = waypoints;
if p.verbose
    fprintf('%s] pianificazione: %s (%.3f s)\n', upper(info.planner), info.message,
info.plan_time);
end
if p.viz.show_planner_debug, plot_planner_debug(env, q0, qf, path, info, p); end
if ~info.success
    warning('Pianificazione fallita: %s. Modificare i parametri del pianificatore.', info.message);
    return;
end

% ---- 2) controllo + simulazione con il modello differenziale
set_seed(p.seed);
sim = simulate_robot(waypoints, q0, qf, env, p);
out.sim = sim;

% ---- 3) analisi
m = compute_metrics(sim, path, waypoints, qf, env, info, p);
out.metrics = m;
out.success = sim.reached && ~m.collision;
if p.verbose
    print_metrics(m);
    if ~sim.reached, warning('Goal non raggiunto entro t_max = %.1f s.', p.sim.t_max); end
end

% ---- 4) grafici e animazione
if p.viz.show_plots, plot_results(sim, path, waypoints, q0, qf, env, m, p); end
if p.viz.animate, animate_robot(sim, path, q0, qf, env, p); end
end
```

A.3 README.md

```
# Pianificazione e controllo di un robot mobile differenziale (MATLAB)

Avvio rapido (MATLAB R2016b o successivo, nessun toolbox richiesto; compatibile anche con GNU Octave
>= 6):

>> cd RobotMobileDifferenziale
>> main % esegue pianificazione + controllo + grafici + animazione

Parametri: tutti in `config/default_params.m`; si sovrascrivono nella sezione
"PARAMETRI UTENTE" di `main.m` oppure da Command Window:

>> addpath(genpath(pwd));
>> p = default_params();
>> p.planner = 'rrt'; p.env.scenario = 'narrow'; p.ctrl.k_rho = 1.5;
>> out = run_project(p);

Test automatici: >> addpath(genpath(pwd)); run_all_tests
Confronto pianificatori: >> T = compare_planners(default_params(), {'cell', 'prm', 'rrt', 'apf'}, 5);

Cartelle: config (parametri, scenari), model (cinematica differenziale, integrazione),
environment (ostacoli, distanze, collisioni), planning (cell decomposition, PRM, RRT, APF),
control (controllore rho-alpha, simulazione), analysis (metriche, confronto),
visualization (grafici, animazione), utils, tests.
```

A.4 config/default_params.m

```
function p = default_params()
```

```

%DEFAULT_PARAMS Restituisce la struttura con TUTTI i parametri del progetto.
%
% p = default_params()
%
% Nessun valore numerico e' cablato nel codice: ogni grandezza usata da
% modello, pianificatori, controllore, simulazione e grafica e' definita
% qui. Per provare valori diversi NON modificare questo file: sovrascrivi
% i campi in main.m (sezione "PARAMETRI UTENTE") oppure da Command Window:
%
% p = default_params();
% p.planner = 'rrt';
% p.ctrl.k_rho = 1.2;
% results = run_project(p);
%
% Unita' di misura: metri, secondi, radianti.

%% ----- ROBOT
p.robot.r = 0.05; % raggio delle ruote r [m]
p.robot.d = 0.30; % distanza tra le ruote (carreggiata) d [m]
p.robot.radius = 0.20; % raggio del cerchio d'ingombro del robot [m]
p.robot.wheel_max = 20; % |omega_R|, |omega_L| massime [rad/s]
p.robot.v_max = 0.8; % velocita' lineare massima [m/s]
p.robot.w_max = 2.5; % velocita' angolare massima [rad/s]

%% ----- AMBIENTE
p.env.scenario = 'base'; % 'base' | 'narrow' | 'trap' | 'custom'
p.env.bounds = []; % [xmin xmax ymin ymax]; [] = quelli dello scenario
p.env.custom_obstacles = {}; % ostacoli se scenario = 'custom' (vedi scenario_library)
p.env.safety_margin = 0.05; % margine aggiunto al raggio del robot [m]

%% ----- CONFIGURAZIONI q0 / qf
p.q0 = []; % [x0 y0 theta0]; [] = configurazione iniziale dello scenario
p.qf = []; % [xf yf thetalf]; [] = configurazione finale dello scenario

%% ----- PIANIFICAZIONE
p.planner = 'prm'; % 'cell' | 'prm' | 'rrt' | 'apf'
p.seed = 1; % seme casuale (riproducibilita' di PRM/RRT/APF)

% 4.1 Cell decomposition con retraction
p.cell.resolution = 0.20; % lato della cella [m]
p.cell.connectivity = 8; % 4 oppure 8 vicini
p.cell.conservative = true; % true: cella libera solo se lo e' tutta
p.cell.clearance_weight = 0.30; % peso [m] che allontana il cammino dagli ostacoli (0 = piu' corto)
p.cell.search = 'dijkstra'; % 'dijkstra' | 'astar'

% 4.2a PRM
p.prm.n_samples = 400; % campioni per tentativo
p.prm.k_neighbors = 10; % vicini massimi tentati per nodo
p.prm.connection_radius = 2.5; % raggio massimo di connessione [m]
p.prm.max_attempts = 4; % tentativi se il grafo non connette q0-qf
p.prm.growth = 1.5; % fattore di crescita dei campioni per tentativo
p.prm.search = 'dijkstra'; % 'dijkstra' | 'astar'

% 4.2b RRT
p.rrt.max_iter = 6000; % iterazioni massime
p.rrt.step = 0.30; % passo di espansione [m]
p.rrt.goal_bias = 0.10; % probabilita' di campionare il goal
p.rrt.goal_tol = 0.30; % distanza per tentare la connessione al goal [m]

% 4.3 Artificial Potential Fields
p.apf.k_att = 1.0; % guadagno attrattivo
p.apf.d_star = 2.0; % soglia quadratico/conico del potenziale attrattivo [m]
p.apf.k_rep = 1.0; % guadagno repulsivo
p.apf.rho0 = 0.8; % raggio d'influenza degli ostacoli [m]
p.apf.goal_rep_n = 2; % correzione GNRON: esponente n (0 = Khatib classico)
p.apf.goal_rep_radius = 1.0; % entro questa distanza dal goal la repulsione e' scalata (d/R)^n [m]
p.apf.step = 0.05; % passo della discesa del gradiente [m]
p.apf.max_iter = 4000; % passi massimi
p.apf.goal_tol = 0.08; % tolleranza di arrivo [m]
p.apf.stall_window = 40; % finestra (passi) per rilevare lo stallo
p.apf.stall_dist = 0.10; % spostamento minimo nella finestra [m]
p.apf.escape = true; % true: fuga dai minimi locali con random walk
p.apf.escape_steps = 40; % passi del primo random walk (raddoppiano a ogni fuga)
p.apf.escape_growth_max = 16; % fattore massimo di crescita del random walk
p.apf.escape_step = 0.10; % lunghezza di un passo di random walk [m]
p.apf.max_escapes = 25; % tentativi massimi di fuga
p.apf.k_virtual = 0.5; % guadagno degli ostacoli virtuali posti sui minimi (0 = disattivi)
p.apf.rho_virtual = 1.0; % raggio d'influenza degli ostacoli virtuali [m]

```

```
% Post-elaborazione comune del percorso
p.post.shortcut      = true;    % semplificazione "string pulling" collision-free
p.post.waypoint_spacing = 0.25; % distanza tra waypoint dopo il ricampionamento [m]
p.post.collision_step = 0.02;  % passo di campionamento nel test dei segmenti [m]

%% ----- CONTROLLO
p.ctrl.k_rho        = 1.0;    % v = k_rho * rho
p.ctrl.k_alpha      = 3.0;    % omega = k_alpha * alpha
p.ctrl.ref_mode     = 'waypoints'; % 'waypoints' | 'continuous'
p.ctrl.switch_radius = 0.25;  % [waypoints] raggio di commutazione [m]
p.ctrl.v_ref        = 0.40;   % [continuous] velocita' del riferimento [m/s]
p.ctrl.max_lag      = 0.50;   % [continuous] ritardo max prima di fermare il riferimento [m]
p.ctrl.cos_alpha_scaling = true; % v moltiplicata per max(cos(alpha),0)
p.ctrl.goal_tol     = 0.05;   % tolleranza di posizione finale [m]
p.ctrl.align_final_heading = true; % ruota sul posto fino a theta_f
p.ctrl.k_theta      = 2.0;    % guadagno dell'allineamento finale
p.ctrl.heading_tol  = 2*pi/180; % tolleranza di orientamento finale [rad]

%% ----- SIMULAZIONE
p.sim.dt            = 0.02;    % passo di integrazione [s]
p.sim.t_max         = 150;     % durata massima [s]
p.sim.integrator    = 'rk4';   % 'euler' | 'rk4' | 'exact'
p.sim.wheel_noise_std = 0.0;   % rumore gaussiano sulle ruote [rad/s] (0 = ideale)

%% ----- VISUALIZZAZIONE
p.viz.show_plots    = true;    % grafici dei risultati
p.viz.show_planner_debug = true; % grafico della struttura del pianificatore
p.viz.animate       = true;    % animazione del moto
p.viz.frame_skip    = 5;      % disegna 1 frame ogni N passi
p.viz.pause         = 0.001;  % pausa tra frame [s]
p.viz.save_gif      = false;   % salva l'animazione come GIF
p.viz.save_figures  = false;   % salva i grafici come PNG
p.viz.output_dir    = 'output';
p.viz.gif_file      = 'robot_motion.gif';

p.verbose = true;          % stampa a console
end
```

A.5 config/scenario_library.m

```
function s = scenario_library(name)
%SCENARIO_LIBRARY Ambienti predefiniti.
%
% s = scenario_library(name) restituisce una struttura con:
% s.bounds [xmin xmax ymin ymax]
% s.obstacles cell array di ostacoli; ogni ostacolo e' una struct
%          creata con make_obstacle:
%          make_obstacle('circle', [xc yc], R)
%          make_obstacle('polygon', [x1 y1; x2 y2; ...])
% s.q0, s.qf configurazioni iniziale e finale suggerite [x y theta]
%
% Scenari: 'base' ostacoli misti circolari e poligonali
%          'narrow' muro con un passaggio stretto
%          'trap' ostacolo a U che genera un minimo locale per APF

circ = @(c, R) make_obstacle('circle', c, R);
poly = @(V) make_obstacle('polygon', V);
rect = @(x1, y1, x2, y2) make_obstacle('polygon', [x1 y1; x2 y1; x2 y2; x1 y2]);

switch lower(name)
case 'base'
    s.bounds = [0 10 0 10];
    s.obstacles = { circ([3.0 3.0], 1.0), ...
                    circ([7.2 2.4], 0.8), ...
                    rect(4.6, 4.8, 6.0, 8.0), ...
                    poly([1.2 6.0; 3.4 6.0; 2.3 8.0]), ...
                    rect(7.0, 6.4, 8.8, 7.1), ...
                    circ([8.2 4.6], 0.55) };
    s.q0 = [0.8 0.8 0];
    s.qf = [9.2 9.2 pi/2];
case 'narrow'
    s.bounds = [0 10 0 10];
    s.obstacles = { rect(4.6, 0.0, 5.3, 4.4), ...
                    rect(4.6, 5.6, 5.3, 10.0), ...
```

```

        circ([2.5 7.0], 0.9), ...
        circ([7.5 3.0], 0.9), ...
        poly([7.0 7.0; 8.6 6.6; 8.2 8.4]) );
    s.q0 = [1.0 2.0 0];
    s.qf = [9.0 8.8 0];
    case 'trap'
        s.bounds = [0 10 0 10];
        s.obstacles = { rect(6.0, 3.0, 6.4, 7.0), ... % fondo della U
                        rect(4.0, 6.6, 6.4, 7.0), ... % braccio superiore
                        rect(4.0, 3.0, 6.4, 3.4) }; % braccio inferiore

        s.q0 = [1.0 5.0 0];
        s.qf = [9.0 5.0 0];
    otherwise
        error('Scenario "%s" sconosciuto. Usa base | narrow | trap | custom.', name);
    end
end

```

A.6 config/validate_params.m

```

function validate_params(p, env, q0, qf)
%VALIDATE_PARAMS Controlla la coerenza dei parametri prima dell'esecuzione.
% Lancia un errore esplicito invece di lasciar fallire il codice a valle.

chk = {p.robot.r, 'robot.r'; p.robot.d, 'robot.d'; p.robot.radius, 'robot.radius';
        p.robot.wheel_max, 'robot.wheel_max'; p.robot.v_max, 'robot.v_max';
        p.robot.w_max, 'robot.w_max'; p.ctrl.k_rho, 'ctrl.k_rho';
        p.ctrl.k_alpha, 'ctrl.k_alpha'; p.sim.dt, 'sim.dt'; p.sim.t_max, 'sim.t_max';
        p.post.waypoint_spacing, 'post.waypoint_spacing'};
for i = 1:size(chk, 1)
    v = chk{i, 1};
    if ~(isnumeric(v) && isscalar(v) && v > 0)
        error('Il parametro p.%s deve essere uno scalare > 0.', chk{i, 2});
    end
end

if ~any(strcmpi(p.planner, {'cell', 'prm', 'rrt', 'apf'}))
    error('p.planner deve essere cell | prm | rrt | apf.');
```

```

end
if ~any(strcmpi(p.ctrl.ref_mode, {'waypoints', 'continuous'}))
    error('p.ctrl.ref_mode deve essere waypoints | continuous.');
```

```

end
if ~any(strcmpi(p.sim.integrator, {'euler', 'rk4', 'exact'}))
    error('p.sim.integrator deve essere euler | rk4 | exact.');
```

```

end
if ~any(p.cell.connectivity == [4 8])
    error('p.cell.connectivity deve essere 4 oppure 8.');
```

```

end
if p.ctrl.k_alpha <= p.ctrl.k_rho
    warning('Si consiglia k_alpha > k_rho per la convergenza del controllore.');
```

```

end

if numel(q0) ~= 3 || numel(qf) ~= 3
    error('q0 e qf devono essere vettori [x y theta].');
```

```

end
if ~is_free(q0(1:2), env)
    error('q0 = %s e'' in collisione o troppo vicina a un ostacolo/bordo.', mat2str(q0, 3));
```

```

end
if ~is_free(qf(1:2), env)
    error('qf = %s e'' in collisione o troppo vicina a un ostacolo/bordo.', mat2str(qf, 3));
```

```

end
end

```

A.7 environment/build_environment.m

```

function [env, q0, qf] = build_environment(p)
%BUILD_ENVIRONMENT Costruisce l'ambiente di lavoro a partire dai parametri.
% env.bounds [xmin xmax ymin ymax]
% env.obstacles cell array di ostacoli (make_obstacle)
% env.inflation raggio robot + margine: distanza minima ammessa
% dal centro del robot a ostacoli e bordi
if strcmpi(p.env.scenario, 'custom')
    if isempty(p.env.bounds), error('Scenario custom: specificare p.env.bounds.');
```

```

        if isempty(p.q0) || isempty(p.qf), error('Scenario custom: specificare p.q0 e p.qf.');
```

```

        s.bounds = p.env.bounds; s.obstacles = p.env.custom_obstacles;
        s.q0 = p.q0; s.qf = p.qf;
    else
        s = scenario_library(p.env.scenario);
    end
    env.bounds = s.bounds;
    if ~isempty(p.env.bounds), env.bounds = p.env.bounds; end
    env.obstacles = s.obstacles;
    env.inflation = p.robot.radius + p.env.safety_margin;
    env.robot_radius = p.robot.radius;

    q0 = s.q0; qf = s.qf;
    if ~isempty(p.q0), q0 = p.q0; end
    if ~isempty(p.qf), qf = p.qf; end
    q0 = q0(:)'; qf = qf(:)';
end
```

A.8 environment/clearance.m

```

function c = clearance(P, env)
%CLEARANCE Distanza con segno dall'ostacolo (o bordo) piu' vicino.
c = min(obstacle_distances(P, env), [], 2);
end
```

A.9 environment/is_free.m

```

function tf = is_free(P, env)
%IS_FREE true se il robot centrato in P non collide (con margine di sicurezza).
tf = clearance(P, env) > env.inflation;
end
```

A.10 environment/is_path_free.m

```

function tf = is_path_free(P, env, step)
%IS_PATH_FREE Verifica che tutti i segmenti della polilinea P siano liberi.
tf = true;
for i = 1:size(P, 1) - 1
    if ~is_segment_free(P(i, :), P(i+1, :), env, step)
        tf = false; return;
    end
end
end
```

A.11 environment/is_segment_free.m

```

function tf = is_segment_free(a, b, env, step)
%IS_SEGMENT_FREE Verifica che il segmento a-b sia libero per il robot.
% Il segmento e' campionato con passo <= step e ogni punto e' verificato.
a = a(1:2); b = b(1:2);
n = max(2, ceil(norm(b - a) / step) + 1);
t = linspace(0, 1, n)';
P = [a(1) + t * (b(1) - a(1)), a(2) + t * (b(2) - a(2))];
tf = all(is_free(P, env));
end
```

A.12 environment/make_obstacle.m

```

function o = make_obstacle(type, a, b)
%MAKE_OBSTACLE Crea un ostacolo geometrico.
% o = make_obstacle('circle', [xc yc], R)
% o = make_obstacle('polygon', V) con V matrice Nx2 dei vertici (ordine qualsiasi
% lungo il perimetro, convesso o concavo)
```

```

switch lower(type)
case 'circle'
    assert(numel(a) == 2 && isscalar(b) && b > 0, 'Cerchio: centro [x y] e raggio > 0');
    o = struct('type', 'circle', 'center', a(:), 'radius', b, 'vertices', []);
case 'polygon'
    assert(size(a, 2) == 2 && size(a, 1) >= 3, 'Poligono: almeno 3 vertici Nx2');
    o = struct('type', 'polygon', 'center', mean(a, 1), 'radius', [], 'vertices', a);
otherwise
    error('Tipo ostacolo "%s" non valido (circle | polygon).', type);
end
end

```

A.13 environment/obstacle_distances.m

```

function [D, Gx, Gy] = obstacle_distances(P, env)
%OBSTACLE_DISTANCES Distanza con segno da ogni ostacolo e da ogni bordo.
%
% [D, Gx, Gy] = obstacle_distances(P, env)
% P : N x 2 punti del piano
% D : N x (M+4) distanze con segno (negativa = punto dentro l'ostacolo);
% le ultime 4 colonne sono i bordi xmin, xmax, ymin, ymax
% Gx,Gy: componenti del gradiente della distanza (versore che si allontana
% dall'ostacolo), usate dai campi di potenziale repulsivi.
N = size(P, 1);
M = numel(env.obstacles);
D = zeros(N, M + 4); Gx = D; Gy = D;
x = P(:, 1); y = P(:, 2);

for j = 1:M
    o = env.obstacles{j};
    if strcmp(o.type, 'circle')
        vx = x - o.center(1); vy = y - o.center(2);
        n = sqrt(vx.^2 + vy.^2);
        D(:, j) = n - o.radius;
        n(n < 1e-12) = 1e-12;
        Gx(:, j) = vx ./ n; Gy(:, j) = vy ./ n;
    else
        V = o.vertices; nv = size(V, 1);
        dmin = inf(N, 1); cx = zeros(N, 1); cy = zeros(N, 1);
        for k = 1:nv
            a = V(k, :); b = V(mod(k, nv) + 1, :);
            ab = b - a; L2 = sum(ab.^2);
            t = ((x - a(1)) * ab(1) + (y - a(2)) * ab(2)) / L2;
            t = min(max(t, 0), 1);
            px = a(1) + t * ab(1); py = a(2) + t * ab(2);
            dk = sqrt((x - px).^2 + (y - py).^2);
            better = dk < dmin;
            dmin(better) = dk(better); cx(better) = px(better); cy(better) = py(better);
        end
        inside = inpolygon(x, y, V(:, 1), V(:, 2));
        sgn = ones(N, 1); sgn(inside) = -1;
        D(:, j) = sgn .* dmin;
        dd = dmin; dd(dd < 1e-12) = 1e-12;
        Gx(:, j) = sgn .* (x - cx) ./ dd;
        Gy(:, j) = sgn .* (y - cy) ./ dd;
    end
end

b = env.bounds;
D(:, M+1) = x - b(1); Gx(:, M+1) = 1;
D(:, M+2) = b(2) - x; Gx(:, M+2) = -1;
D(:, M+3) = y - b(3); Gy(:, M+3) = 1;
D(:, M+4) = b(4) - y; Gy(:, M+4) = -1;
end

```

A.14 model/diffdrive_dynamics.m

```

function qdot = diffdrive_dynamics(q, wR, wL, robot)
%DIFFDRIVE_DYNAMICS Modello cinematico differenziale qdot = f(q, wR, wL).
% q = [x y theta]; xdot = v cos(theta), ydot = v sin(theta), thetadot = w
[v, w] = wheels_to_unicycle(wR, wL, robot);
qdot = [v * cos(q(3)), v * sin(q(3)), w];

```

```
end
```

A.15 model/integrate_step.m

```
function qn = integrate_step(q, wR, wL, robot, dt, method)
%INTEGRATE_STEP Avanza lo stato di un passo dt con ingressi ruota costanti.
% method: 'euler' (Eulero esplicito), 'rk4' (Runge-Kutta 4) oppure
%          'exact' (integrazione in forma chiusa lungo un arco di circonferenza).
f = @(qq) diffdrive_dynamics(qq, wR, wL, robot);
switch lower(method)
case 'euler'
    qn = q + dt * f(q);
case 'rk4'
    k1 = f(q);
    k2 = f(q + dt/2 * k1);
    k3 = f(q + dt/2 * k2);
    k4 = f(q + dt * k3);
    qn = q + dt/6 * (k1 + 2*k2 + 2*k3 + k4);
case 'exact'
    [v, w] = wheels_to_unicycle(wR, wL, robot);
    th = q(3);
    if abs(w) < 1e-9
        qn = [q(1) + v*dt*cos(th), q(2) + v*dt*sin(th), th];
    else
        qn = [q(1) + v/w * (sin(th + w*dt) - sin(th)), ...
              q(2) - v/w * (cos(th + w*dt) - cos(th)), ...
              th + w*dt];
    end
otherwise
    error('Integratore "%s" non valido.', method);
end
qn(3) = wrap_to_pi(qn(3));
end
```

A.16 model/saturate_wheels.m

```
function [wR, wL, scale] = saturate_wheels(wR, wL, wmax)
%SATURATE_WHEELS Saturazione delle ruote che preserva la curvatura.
% Se una ruota supera wmax, entrambe sono scalate dello stesso fattore:
% il rapporto wR/wL (e quindi la curvatura w/v del percorso) resta invariato.
m = max(abs(wR), abs(wL));
scale = 1;
if m > wmax
    scale = wmax / m;
    wR = wR * scale;
    wL = wL * scale;
end
end
```

A.17 model/unicycle_to_wheels.m

```
function [wR, wL] = unicycle_to_wheels(v, w, robot)
%UNICYCLE_TO_WHEELS Cinematica inversa: (v, w) -> velocità angolari delle ruote.
% wR = (2v + w d) / (2r), wL = (2v - w d) / (2r)
wR = (2 .* v + w .* robot.d) ./ (2 * robot.r);
wL = (2 .* v - w .* robot.d) ./ (2 * robot.r);
end
```

A.18 model/wheels_to_unicycle.m

```
function [v, w] = wheels_to_unicycle(wR, wL, robot)
%WHEELS_TO_UNICYCLE Cinematica diretta del robot differenziale.
% v = r/2 (wR + wL), w = r/d (wR - wL)
v = robot.r / 2 .* (wR + wL);
w = robot.r / robot.d .* (wR - wL);
```

```
end
```

A.19 planning/apf_potential.m

```
function [U, gx, gy] = apf_potential(P, goal, env, prm, virt)
%APF_POTENTIAL Potenziale artificiale totale e suo gradiente.
%
% Attrattivo (quadratico vicino al goal, conico lontano):
% d <= d*: Ua = 1/2 ka d^2 grad = ka (p - pg)
% d > d*: Ua = d* ka d - 1/2 ka d^2 grad = d* ka (p - pg)/d
% Repulsivo (Khatib) per ogni ostacolo e bordo, con rho = clearance - inflation:
% rho <= rho0: Ur = 1/2 kr (1/rho - 1/rho0)^2
% grad Ur = -kr (1/rho - 1/rho0) / rho^2 * grad(rho)
% Correzione GNRON (Ge & Cui): se d < R = goal_rep_radius il potenziale
% repulsivo e' moltiplicato per f = (d/R)^n, cosi' il goal resta il minimo
% globale anche quando e' vicino a ostacoli o bordi:
% grad(f Ur) = f grad(Ur) + Ur grad(f), grad(f) = n d^(n-2) (p - pg) / R^n
% Ostacoli virtuali (opzionali, virt = K x 2 punti dei minimi locali gia'
% visitati): stesso potenziale di Khatib con guadagno k_virtual e raggio
% rho_virtual, usato per "riempire" i pozzi dei minimi locali.
% P: N x 2 punti; U: N x 1; gx, gy: componenti del gradiente.
if nargin < 5, virt = zeros(0, 2); end
dx = P(:, 1) - goal(1); dy = P(:, 2) - goal(2);
d = sqrt(dx.^2 + dy.^2);
ka = prm.k_att; ds = prm.d_star;
near = d <= ds;
U = zeros(size(d)); gx = U; gy = U;
U(near) = 0.5 * ka * d(near).^2;
gx(near) = ka * dx(near); gy(near) = ka * dy(near);
far = ~near;
U(far) = ds * ka * d(far) - 0.5 * ka * ds^2;
gx(far) = ds * ka * dx(far) ./ d(far);
gy(far) = ds * ka * dy(far) ./ d(far);

[D, Gx, Gy] = obstacle_distances(P, env);
rho = max(D - env.inflation, 1e-3);
act = rho <= prm.rho0;
inv = (1 ./ rho - 1 / prm.rho0) .* act;
Ur = sum(0.5 * prm.k_rep * inv.^2, 2);
coef = -prm.k_rep * inv ./ rho.^2;
grx = sum(coef .* Gx, 2);
gry = sum(coef .* Gy, 2);
f = ones(size(d)); dfx = zeros(size(d)); dfy = dfx;
if prm.goal_rep_n > 0
    R = prm.goal_rep_radius; n = prm.goal_rep_n;
    in = d < R;
    f(in) = (d(in) / R).^n;
    dd = max(d(in), 1e-9);
    dfx(in) = n * dd.^(n - 2) .* dx(in) / R^n;
    dfy(in) = n * dd.^(n - 2) .* dy(in) / R^n;
end
U = U + f .* Ur;
gx = gx + f .* grx + Ur .* dfx;
gy = gy + f .* gry + Ur .* dfy;

if ~isempty(virt) && prm.k_virtual > 0
    for k = 1:size(virt, 1)
        vx = P(:, 1) - virt(k, 1); vy = P(:, 2) - virt(k, 2);
        r = max(sqrt(vx.^2 + vy.^2), 1e-3);
        a = r <= prm.rho_virtual;
        iv = (1 ./ r - 1 / prm.rho_virtual) .* a;
        U = U + 0.5 * prm.k_virtual * iv.^2;
        cf = -prm.k_virtual * iv ./ r.^2;
        gx = gx + cf .* vx ./ r;
        gy = gy + cf .* vy ./ r;
    end
end
end
end
```

A.20 planning/graph_search.m

```
function [nodes, cost] = graph_search(nbrs, costs, s, t, method, coords)
```

```
%GRAPH_SEARCH Cammino minimo su grafo pesato (Dijkstra o A*).
%
% [nodes, cost] = graph_search(nbrs, costs, s, t, method, coords)
% nbrs{i} : indici dei vicini del nodo i
% costs{i} : costi (>= 0) dei rispettivi archi
% s, t : nodo di partenza e di arrivo
% method : 'dijkstra' (h = 0) oppure 'astar' (h = distanza euclidea)
% coords : N x 2 coordinate (usate solo da A*)
% nodes : sequenza di nodi da s a t ([]) se t non raggiungibile)
N = numel(nbrs);
g = inf(N, 1); g(s) = 0;
prev = zeros(N, 1);
closed = false(N, 1);
h = zeros(N, 1);
if strcmpi(method, 'astar')
    h = sqrt(sum((coords - coords(t, :)).^2, 2));
end

while true
    f = g + h; f(closed) = inf;
    [fmin, u] = min(f);
    if isinf(fmin) || u == t, break; end
    closed(u) = true;
    nb = nbrs{u}(:); c = costs{u}(:);
    if isempty(nb), continue; end
    alt = g(u) + c;
    upd = alt < g(nb) & ~closed(nb);
    g(nb(upd)) = alt(upd);
    prev(nb(upd)) = u;
end

cost = g(t);
if isinf(cost), nodes = []; return; end
nodes = t;
while nodes(1) ~= s
    nodes = [prev(nodes(1)); nodes]; %#ok<AGROW>
end
end
```

A.21 planning/plan_apf.m

```
function [path, info] = plan_apf(env, q0, qf, prm, coll_step)
%PLAN_APF Pianificazione con campi di potenziale artificiali.
% Discesa del gradiente a passo normalizzato:  $p_{k+1} = p_k - \text{step} * \text{grad}U/|\text{grad}U|$ 
% Minimo locale: rilevato quando lo spostamento negli ultimi stall_window
% passi e' minore di stall_dist (o il gradiente e' quasi nullo). Se
% prm.escape = true:
% - il minimo viene marcato con un ostacolo virtuale repulsivo (riempie il
% pozzo, cosi' la discesa successiva non vi ricade);
% - si esegue un random walk collision-free la cui lunghezza raddoppia a
% ogni fuga (fino a escape_growth_max volte), poi si riprende la discesa.
% Con prm.escape = false la pianificazione fallisce e il minimo e' segnalato.
goal = qf(1:2); p = q0(1:2);
cap = prm.max_iter + prm.max_escapes * prm.escape_steps * prm.escape_growth_max + 2;
path = zeros(cap, 2); n = 1; path(1, :) = p;
minima = zeros(0, 2); n_esc = 0; success = false; last_stall = 0;
for it = 1:prm.max_iter
    if norm(p - goal) < prm.goal_tol
        n = n + 1; path(n, :) = goal; success = true; break;
    end
    [~, gx, gy] = apf_potential(p, goal, env, prm, minima);
    g = [gx gy]; ng = norm(g);
    stalled = ng < 1e-6;
    if ~stalled && n > prm.stall_window && n - last_stall > prm.stall_window
        stalled = norm(path(n, :) - path(n - prm.stall_window, :)) < prm.stall_dist;
    end
    if stalled
        minima(end+1, :) = p; %#ok<AGROW>
        if ~prm.escape || n_esc >= prm.max_escapes, break; end
        n_esc = n_esc + 1;
        nsteps = prm.escape_steps * min(2^(n_esc - 1), prm.escape_growth_max);
        for k = 1:nsteps
            % random walk di fuga
            th = 2 * pi * rand;
            pn = p + prm.escape_step * [cos(th) sin(th)];
            if is_segment_free(p, pn, env, coll_step)
```

```

        p = pn; n = n + 1; path(n, :) = p;
    end
    end
    last_stall = n;
    continue;
end
h = min(prm.step, norm(p - goal));
pn = p - h * g / ng;
tries = 0;
while ~is_segment_free(p, pn, env, coll_step) && tries < 6
    h = h / 2; pn = p - h * g / ng; tries = tries + 1;
end
if tries == 6, minima(end+1, :) = p; break; end %#ok<AGROW>
p = pn; n = n + 1; path(n, :) = p;
end
raw = path(1:n, :);
info.raw_path = raw; info.local_minima = minima; info.escapes = n_esc;
info.iterations = it; info.goal = goal;
if ~success
    path = []; info.success = false;
    if isempty(minima)
        info.message = 'max_iter raggiunto';
    else
        info.message = sprintf('bloccato in un minimo locale in (%.2f, %.2f)', minima(end, 1),
minima(end, 2));
    end
    return;
end
path = raw; info.success = true; info.message = 'ok';
end

```

A.22 planning/plan_cell_decomposition.m

```

function [path, info] = plan_cell_decomposition(env, q0, qf, prm)
%PLAN_CELL_DECOMPOSITION Decomposizione in celle + retraction + ricerca su grafo.
%
% 1) Il rettangolo di lavoro e' diviso in celle quadrate di lato prm.resolution.
% 2) Una cella e' LIBERA se la sua clearance supera il raggio gonfiato del
%    robot (+ mezza diagonale della cella se prm.conservative = true).
% 3) Si costruisce il grafo di adiacenza delle celle libere (4 o 8 vicini;
%    con 8 vicini le diagonali sono ammesse solo senza "tagliare l'angolo").
% 4) Retraction: ogni cella libera e' ridotta al suo nodo centrale; il costo
%    di un arco e' la lunghezza moltiplicata per (1 + w/clearance) cosi' che
%    il cammino sia "ritratto" verso il centro dello spazio libero.
% 5) Dijkstra (o A*) dal nodo-cella di q0 a quello di qf.
res = prm.resolution; b = env.bounds;
nx = floor((b(2) - b(1)) / res); ny = floor((b(4) - b(3)) / res);
xc = b(1) + ((1:nx) - 0.5) * res;
yc = b(3) + ((1:ny) - 0.5) * res;
[X, Y] = meshgrid(xc, yc);
C = [X(:) Y(:)];
cl = clearance(C, env);
thr = env.inflation;
if prm.conservative, thr = thr + res * sqrt(2) / 2; end
free = cl > thr;
F = reshape(free, ny, nx);

if prm.connectivity == 4
    off = [1 0; -1 0; 0 1; 0 -1];
else
    off = [1 0; -1 0; 0 1; 0 -1; 1 1; 1 -1; -1 1; -1 -1];
end
w = prm.clearance_weight;
Ncell = nx * ny;
nbrs = cell(Ncell, 1); costs = cell(Ncell, 1);
ids = find(free);
for k = 1:numel(ids)
    i = ids(k);
    iy = mod(i - 1, ny) + 1; ix = floor((i - 1) / ny) + 1;
    nb = zeros(1, 8); cc = zeros(1, 8); m = 0;
    for o = 1:size(off, 1)
        jy = iy + off(o, 1); jx = ix + off(o, 2);
        if jy < 1 || jy > ny || jx < 1 || jx > nx || ~F(jy, jx), continue; end
        if off(o, 1) ~= 0 && off(o, 2) ~= 0 && (~F(jy, ix) || ~F(iy, jx)), continue; end
        j = jy + (jx - 1) * ny;
    end
end

```

```

        L = res * norm(off(o, :));
        cavg = max(0.5 * (cl(i) + cl(j)) - env.inflation, 1e-3);
        m = m + 1; nb(m) = j; cc(m) = L * (1 + w / cavg);
    end
    nbrs{i} = nb(1:m); costs{i} = cc(1:m);
end

s = nearest_visible_cell(q0(1:2), C, ids, env);
t = nearest_visible_cell(qf(1:2), C, ids, env);
info.grid = struct('xc', xc, 'yc', yc, 'free', F, 'res', res);
info.n_free = numel(ids); info.n_cells = Ncell;
if isempty(s) || isempty(t)
    path = []; info.success = false;
    info.message = 'q0 o qf non collegabili a una cella libera'; return;
end
[nodes, cost] = graph_search(nbrs, costs, s, t, prm.search, C);
if isempty(nodes)
    path = []; info.success = false;
    info.message = 'nessun cammino nel grafo di adiacenza'; return;
end
info.retracted_nodes = C(nodes, :);
info.cost = cost; info.success = true; info.message = 'ok';
path = [q0(1:2); C(nodes, :); qf(1:2)];
end

function c = nearest_visible_cell(p, C, ids, env)
% cella libera piu' vicina raggiungibile in linea retta da p
d = sqrt(sum((C(ids, :) - p).^2, 2));
[~, ord] = sort(d);
c = [];
for k = 1:min(50, numel(ord))
    cand = ids(ord(k));
    if is_segment_free(p, C(cand, :), env, 0.02)
        c = cand; return;
    end
end
end
end

```

A.23 planning/plan_path.m

```

function [path, waypoints, info] = plan_path(env, q0, qf, p)
%PLAN_PATH Esegue il pianificatore scelto e post-elabora il percorso.
% path      : percorso geometrico pianificato (dopo shortcut se attivo)
% waypoints : percorso ricampionato a passo p.post.waypoint_spacing
% info      : diagnostica del pianificatore (+ tempo, lunghezze)
set_seed(p.seed);
cs = p.post.collision_step;
t0 = tic;
switch lower(p.planner)
    case 'cell', [raw, info] = plan_cell_decomposition(env, q0, qf, p.cell);
    case 'prm',  [raw, info] = plan_prm(env, q0, qf, p.prm, cs);
    case 'rrt',  [raw, info] = plan_rrt(env, q0, qf, p.rrt, cs);
    case 'apf',  [raw, info] = plan_apf(env, q0, qf, p.apf, cs);
end
info.planner = lower(p.planner);
if ~info.success
    info.plan_time = toc(t0);
    path = []; waypoints = [];
    return;
end
info.raw = raw;
info.raw_length = path_length(raw);
path = raw;
if p.post.shortcut, path = shortcut_path(raw, env, cs); end
waypoints = resample_path(path, p.post.waypoint_spacing);
info.plan_time = toc(t0);
info.length = path_length(path);
end

```

A.24 planning/plan_prm.m

```

function [path, info] = plan_prm(env, q0, qf, prm, coll_step)

```

```

%PLAN_PRM Probabilistic RoadMap.
% 1) Campiona n configurazioni uniformi e tiene quelle collision-free.
% 2) Collega ogni nodo ai k vicini entro connection_radius con archi liberi.
% 3) Cerca il cammino q0 -> qf (Dijkstra / A*).
% Se q0 e qf restano in componenti diverse, ripete con piu' campioni.
b = env.bounds; m = env.inflation;
for attempt = 1:prm.max_attempts
    n = round(prm.n_samples * prm.growth^(attempt - 1));
    S = [b(1) + m + rand(n, 1) * (b(2) - b(1) - 2*m), ...
        b(3) + m + rand(n, 1) * (b(4) - b(3) - 2*m)];
    S = S(is_free(S, env), :);
    V = [q0(1:2); qf(1:2); S];
    N = size(V, 1);
    nbrs = cell(N, 1); costs = cell(N, 1);
    tested = false(N);
    E = zeros(0, 2);
    for i = 1:N
        d = sqrt(sum((V - V(i, :)).^2, 2)); d(i) = inf;
        [ds, ord] = sort(d);
        ord = ord(ds <= prm.connection_radius);
        ord = ord(1:min(prm.k_neighbors, numel(ord)));
        for j = ord'
            if tested(i, j), continue; end
            tested(i, j) = true; tested(j, i) = true;
            if is_segment_free(V(i, :), V(j, :), env, coll_step)
                nbrs{i}(end+1) = j; costs{i}(end+1) = d(j);
                nbrs{j}(end+1) = i; costs{j}(end+1) = d(j);
                E(end+1, :) = [i j]; %#ok<AGROW>
            end
        end
    end
    nodes = graph_search(nbrs, costs, 1, 2, prm.search, V);
    if ~isempty(nodes), break; end
end
info.nodes = V; info.edges = E; info.attempts = attempt;
info.n_nodes = N; info.n_edges = size(E, 1);
if isempty(nodes)
    path = []; info.success = false;
    info.message = 'q0 e qf in componenti non connesse della roadmap'; return;
end
path = V(nodes, :); info.success = true; info.message = 'ok';
end

```

A.25 planning/plan_rrt.m

```

function [path, info] = plan_rrt(env, q0, qf, prm, coll_step)
%PLAN_RRT Rapidly-exploring Random Tree.
% Ad ogni iterazione campiona un punto (il goal con probabilita' goal_bias),
% trova il nodo piu' vicino dell'albero, avanza di 'step' verso il campione
% e aggiunge il nuovo nodo se il segmento e' libero. Termina quando un nodo
% entro goal_tol vede il goal.
b = env.bounds; m = env.inflation; goal = qf(1:2);
Nmax = prm.max_iter + 2;
V = zeros(Nmax, 2); parent = zeros(Nmax, 1);
V(1, :) = q0(1:2); n = 1; success = false;
for it = 1:prm.max_iter
    if rand < prm.goal_bias
        xr = goal;
    else
        xr = [b(1) + m + rand * (b(2) - b(1) - 2*m), b(3) + m + rand * (b(4) - b(3) - 2*m)];
    end
    [~, in] = min(sum((V(1:n, :) - xr).^2, 2));
    dir = xr - V(in, :); L = norm(dir);
    if L < 1e-9, continue; end
    xn = V(in, :) + dir / L * min(prm.step, L);
    if ~is_segment_free(V(in, :), xn, env, coll_step), continue; end
    n = n + 1; V(n, :) = xn; parent(n) = in;
    if norm(xn - goal) <= prm.goal_tol && is_segment_free(xn, goal, env, coll_step)
        n = n + 1; V(n, :) = goal; parent(n) = n - 1;
        success = true; break;
    end
end
V = V(1:n, :); parent = parent(1:n);
info.nodes = V; info.parent = parent; info.iterations = it; info.n_nodes = n;
if ~success

```

```

    path = []; info.success = false;
    info.message = 'goal non raggiunto entro max_iter'; return;
end
idx = n;
while parent(idx(1)) ~= 0, idx = [parent(idx(1)); idx]; end %#ok<AGROW>
path = V(idx, :); info.success = true; info.message = 'ok';
end

```

A.26 planning/resample_path.m

```

function W = resample_path(P, spacing)
%RESAMPLE_PATH Ricampiona la polilinea P con waypoint a distanza <= spacing.
% Ogni segmento e' suddiviso in ceil(L/spacing) parti uguali e i vertici
% originali sono SEMPRE conservati: il percorso ricampionato coincide con
% quello pianificato (nessun "taglio" degli spigoli), quindi resta
% collision-free.
seg = sqrt(sum(diff(P, 1, 1).^2, 2));
P = P([true; seg > 1e-9], :);
if size(P, 1) < 2, W = P; return; end
W = P(1, :);
for i = 1:size(P, 1) - 1
    a = P(i, :); b = P(i + 1, :);
    n = max(1, ceil(norm(b - a) / spacing));
    t = (1:n)' / n;
    W = [W; a + t * (b - a)]; %#ok<AGROW>
end
end

```

A.27 planning/shortcut_path.m

```

function P = shortcut_path(P, env, step)
%SHORTCUT_PATH Semplificazione "string pulling" di un percorso collision-free.
% Da ogni punto i salta al punto j piu' lontano raggiungibile in linea retta
% (scansione in avanti): elimina zig-zag di PRM/RRT/APF e i punti allineati
% della griglia, mantenendo il percorso libero da collisioni.
if size(P, 1) <= 2, return; end
keep = 1; i = 1; n = size(P, 1);
while i < n
    j = i + 1;
    while j < n && is_segment_free(P(i, :), P(j + 1, :), env, step)
        j = j + 1;
    end
    keep(end+1) = j; %#ok<AGROW>
    i = j;
end
P = P(keep, :);
end

```

A.28 control/controller_rho_alpha.m

```

function [v, w, rho, alpha] = controller_rho_alpha(q, target, ctrl)
%CONTROLLER_RHO_ALPHA Controllore cinematico polare verso un punto obiettivo.
% rho = distanza tra robot e target
% alpha = angolo tra l'asse del robot e la direzione del target, in (-pi, pi]
% v = k_rho * rho, w = k_alpha * alpha
% Con ctrl.cos_alpha_scaling = true: v = k_rho * rho * max(cos(alpha), 0)
% (il robot rallenta mentre si orienta e non avanza se il target e' dietro).
dx = target(1) - q(1); dy = target(2) - q(2);
rho = sqrt(dx^2 + dy^2);
alpha = wrap_to_pi(atan2(dy, dx) - q(3));
v = ctrl.k_rho * rho;
if ctrl.cos_alpha_scaling, v = v * max(cos(alpha), 0); end
w = ctrl.k_alpha * alpha;
end

```

A.29 control/point_on_path.m

```
function pt = point_on_path(P, s_cum, s)
%POINT_ON_PATH Punto alla ascissa curvilinea s lungo la polilinea P.
s = min(max(s, 0), s_cum(end));
pt = [interp1(s_cum, P(:, 1), s), interp1(s_cum, P(:, 2), s)];
end
```

A.30 control/simulate_robot.m

```
function res = simulate_robot(waypoints, q0, qf, env, p)
%SIMULATE_ROBOT Inseguimento del percorso con il modello differenziale.
%
% Ad ogni passo dt:
% 1) si sceglie il target (waypoint corrente o punto di riferimento continuo);
% 2) il controllore calcola (v, w) = (k_rho rho, k_alpha alpha);
% 3) (v, w) vengono limitati e convertiti in (omega_R, omega_L);
% 4) le ruote vengono saturate (preservando la curvatura) e, se richiesto,
%    disturbate con rumore gaussiano;
% 5) lo stato viene integrato con il modello cinematico differenziale.
% Fasi: 1 = inseguimento, 2 = allineamento finale a theta_f, 3 = arrivato.
c = p.ctrl; rb = p.robot; dt = p.sim.dt;
N = ceil(p.sim.t_max / dt);
q = zeros(N + 1, 3); q(1, :) = q0;
[t, wR, wL, v, w, rho, alpha, phase_log] = deal(zeros(N + 1, 1));
target = zeros(N + 1, 2);
nwp = size(waypoints, 1);
s_cum = [0; cumsum(sqrt(sum(diff(waypoints, 1, 1).^2, 2)))]';
L = s_cum(end);
idx = min(2, nwp); s_ref = 0; phase = 1;
goal = qf(1:2);
continuous = strcmpi(c.ref_mode, 'continuous');
k = 1;
for k = 1:N
    pos = q(k, 1:2);
    if phase == 1
        if continuous
            if norm(point_on_path(waypoints, s_cum, s_ref) - pos) < c.max_lag
                s_ref = min(s_ref + c.v_ref * dt, L);
            end
            tg = point_on_path(waypoints, s_cum, s_ref);
            at_end = s_ref >= L;
        else
            while idx < nwp && norm(waypoints(idx, :) - pos) < c.switch_radius
                idx = idx + 1;
            end
            tg = waypoints(idx, :);
            at_end = idx == nwp;
        end
        if at_end && norm(goal - pos) < c.goal_tol
            if c.align_final_heading, phase = 2; else, phase = 3; end
        end
    end
    if phase == 1
        [vc, wc, rho(k), alpha(k)] = controller_rho_alpha(q(k, :), tg, c);
    elseif phase == 2
        tg = goal;
        e = wrap_to_pi(qf(3) - q(k, 3));
        vc = 0; wc = c.k_theta * e;
        rho(k) = norm(goal - pos); alpha(k) = e;
        if abs(e) < c.heading_tol, phase = 3; end
    end
    target(k, :) = tg; t(k) = (k - 1) * dt; phase_log(k) = phase;
    if phase == 3
        rho(k) = norm(goal - pos); alpha(k) = wrap_to_pi(qf(3) - q(k, 3));
        break;
    end
    % limiti di velocita' del veicolo
    vc = min(max(vc, -rb.v_max), rb.v_max);
    wc = min(max(wc, -rb.w_max), rb.w_max);
    % (v, w) -> (omega_R, omega_L) + saturazione + rumore
    [r_, l_] = unicycle_to_wheels(vc, wc, rb);
    [r_, l_] = saturate_wheels(r_, l_, rb.wheel_max);
    if p.sim.wheel_noise_std > 0
        r_ = r_ + p.sim.wheel_noise_std * randn;
    end
```

```

        l_ = l_ + p.sim.wheel_noise_std * randn;
    end
    wR(k) = r_; wL(k) = l_;
    [v(k), w(k)] = wheels_to_unicycle(r_, l_, rb);
    q(k + 1, :) = integrate_step(q(k, :), r_, l_, rb, dt, p.sim.integrator);
end
if phase ~= 3 % tempo massimo raggiunto: registra l'ultimo stato
    k = N + 1; t(k) = N * dt; target(k, :) = target(k - 1, :);
    rho(k) = norm(goal - q(k, 1:2)); phase_log(k) = phase;
end
K = k;
res.t = t(1:K); res.q = q(1:K, :);
res.wR = wR(1:K); res.wL = wL(1:K); res.v = v(1:K); res.w = w(1:K);
res.rho = rho(1:K); res.alpha = alpha(1:K); res.target = target(1:K, :);
res.phase = phase_log(1:K);
res.reached = phase == 3;
res.t_final = res.t(end);
end

```

A.31 analysis/compare_planners.m

```

function T = compare_planners(p, planners, n_runs)
%COMPARE_PLANNERS Confronta i pianificatori sullo stesso scenario.
%
% T = compare_planners(p)          tutti i metodi, 1 esecuzione
% T = compare_planners(p, {'prm','rrt'}, 10) 10 esecuzioni (semi diversi)
%
% Per i metodi stocastici (PRM, RRT, APF con fuga) si usano i semi
% p.seed, p.seed+1, ...; le metriche riportate sono medie sulle esecuzioni
% riuscite. T e' una struct array con una riga per metodo.
if nargin < 2 || isempty(planners), planners = {'cell', 'prm', 'rrt', 'apf'}; end
if nargin < 3, n_runs = 1; end
p.viz.show_plots = false; p.viz.animate = false;
p.viz.show_planner_debug = false; p.verbose = false;
T = struct([]);
for i = 1:numel(planners)
    acc = []; ok = 0;
    for r = 1:n_runs
        pp = p; pp.planner = planners{i}; pp.seed = p.seed + r - 1;
        out = run_project(pp);
        if out.success
            ok = ok + 1;
            m = out.metrics;
            row = [m.plan_time m.planned_length m.time_to_goal m.ct_rms m.ct_max ...
                  m.min_clear_exec m.final_pos_error m.max_wheel];
            acc = [acc; row]; %#ok<AGROW>
        end
    end
    T(i).planner = planners{i};
    T(i).success_rate = ok / n_runs;
    if ok > 0
        mu = mean(acc, 1);
    else
        mu = nan(1, 8);
    end
    T(i).plan_time = mu(1); T(i).length = mu(2); T(i).time_to_goal = mu(3);
    T(i).ct_rms = mu(4); T(i).ct_max = mu(5); T(i).min_clear = mu(6);
    T(i).final_err = mu(7); T(i).max_wheel = mu(8);
end
fprintf('\nConfronto pianificatori - scenario "%s" (%d esecuzioni per metodo)\n', p.env.scenario,
n_runs);
fprintf('%-6s %7s %9s %9s %9s %9s %9s %9s %10s\n', 'Metodo', 'Succ.', 't_plan[s]', ...
        'L[m]', 't_goal[s]', 'eRMS[m]', 'eMax[m]', 'clr[m]', 'wMax[r/s]');
for i = 1:numel(T)
    fprintf('%-6s %6.0f%% %9.3f %9.2f %9.2f %9.4f %9.4f %9.3f %10.2f\n', upper(T(i).planner), ...
        100 * T(i).success_rate, T(i).plan_time, T(i).length, T(i).time_to_goal, ...
        T(i).ct_rms, T(i).ct_max, T(i).min_clear, T(i).max_wheel);
end
end

```

A.32 analysis/compute_metrics.m

```
function m = compute_metrics(res, path, waypoints, qf, env, info, p)
%COMPUTE_METRICS Indicatori quantitativi di pianificazione e controllo.
Q = res.q(:, 1:2);
ct = point_to_path_distance(Q, waypoints);
m.planner = info.planner;
m.plan_time = info.plan_time;
m.planned_length = path_length(path);
m.executed_length = path_length(Q);
m.reached = res.reached;
m.time_to_goal = res.t_final;
m.final_pos_error = norm(Q(end, :) - qf(1:2));
m.final_head_error = abs(wrap_to_pi(res.q(end, 3) - qf(3)));
m.cross_track = ct;
m.ct_rms = sqrt(mean(ct.^2));
m.ct_max = max(ct);
cl_plan = clearance(resample_path(path, 0.02), env) - env.robot_radius;
cl_exec = clearance(Q, env) - env.robot_radius;
m.min_clear_plan = min(cl_plan); % distanza minima bordo-robot / ostacolo
m.min_clear_exec = min(cl_exec);
m.collision = any(cl_exec <= 0);
m.max_wheel = max(max(abs(res.wR)), max(abs(res.wL)));
m.saturated_frac = mean(max(abs(res.wR), abs(res.wL)) >= p.robot_wheel_max - 1e-9);
m.wheel_effort = sum((res.wR.^2 + res.wL.^2) * p.sim.dt);
m.path_efficiency = m.planned_length / max(m.executed_length, eps);
end
```

A.33 analysis/point_to_path_distance.m

```
function d = point_to_path_distance(Q, P)
%POINT_TO_PATH_DISTANCE Distanza minima di ogni punto Q(i,:) dalla polilinea P.
% E' l'errore di tracking laterale (cross-track error).
n = size(Q, 1); d = inf(n, 1);
if size(P, 1) == 1
    d = sqrt(sum((Q - P).^2, 2)); return;
end
for k = 1:size(P, 1) - 1
    a = P(k, :); ab = P(k + 1, :) - a; L2 = max(sum(ab.^2), 1e-12);
    tt = min(max(((Q(:, 1) - a(1)) * ab(1) + (Q(:, 2) - a(2)) * ab(2)) / L2, 0), 1);
    dk = sqrt((Q(:, 1) - a(1) - tt * ab(1)).^2 + (Q(:, 2) - a(2) - tt * ab(2)).^2);
    d = min(d, dk);
end
end
```

A.34 analysis/print_metrics.m

```
function print_metrics(m)
%PRINT_METRICS Stampa a console le metriche di una simulazione.
fprintf('\n----- RISULTATI (%s) ----- \n', upper(m.planner));
fprintf('Tempo di pianificazione : %8.3f s\n', m.plan_time);
fprintf('Lunghezza percorso pianificato: %8.3f m\n', m.planned_length);
fprintf('Lunghezza percorso eseguito : %8.3f m\n', m.executed_length);
fprintf('Goal raggiunto : %8s\n', tf2str(m.reached));
fprintf('Tempo di esecuzione : %8.2f s\n', m.time_to_goal);
fprintf('Errore finale di posizione : %8.4f m\n', m.final_pos_error);
fprintf('Errore finale di orientamento : %8.3f deg\n', m.final_head_error * 180/pi);
fprintf('Errore di tracking RMS / max : %8.4f / %.4f m\n', m.ct_rms, m.ct_max);
fprintf('Distanza min. dagli ostacoli : %8.3f m (pianificato) / %.3f m (eseguito)\n', ...
    m.min_clear_plan, m.min_clear_exec);
fprintf('Collisioni : %8s\n', tf2str(m.collision));
fprintf('|omega ruota| massima : %8.2f rad/s (saturazione %.1f%% del tempo)\n', ...
    m.max_wheel, 100 * m.saturated_frac);
fprintf('----- \n');
end
function s = tf2str(b)
if b, s = 'SI'; else, s = 'NO'; end
end
```

A.35 visualization/animate_robot.m

```
function animate_robot(res, path, q0, qf, env, p)
%ANIMATE_ROBOT Animazione del moto (corpo, ruote, prua e scia).
% Con p.viz.save_gif = true salva una GIF in p.viz.output_dir.
fig = figure('Name', 'Animazione', 'Color', 'w'); ax = gca;
plot_environment(env, ax);
plot(ax, path(:, 1), path(:, 2), 'b--', 'LineWidth', 1.2);
plot_pose(ax, q0, [0 0.7 0], 'q_0'); plot_pose(ax, qf, [0.85 0 0], 'q_f');
S = robot_shape(res.q(1, :), p.robot);
hB = fill(ax, S.body(:, 1), S.body(:, 2), [0.3 0.6 1], 'FaceAlpha', 0.6);
hR = fill(ax, S.wheelR(:, 1), S.wheelR(:, 2), 'k');
hL = fill(ax, S.wheelL(:, 1), S.wheelL(:, 2), 'k');
hH = plot(ax, S.head(:, 1), S.head(:, 2), 'r-', 'LineWidth', 2);
hT = plot(ax, res.q(1, 1), res.q(1, 2), 'r-', 'LineWidth', 1.5);
hG = plot(ax, res.target(1, 1), res.target(1, 2), 'g+', 'MarkerSize', 10, 'LineWidth', 2);
gif = '';
if p.viz.save_gif
    if ~exist(p.viz.output_dir, 'dir'), mkdir(p.viz.output_dir); end
    gif = fullfile(p.viz.output_dir, p.viz.gif_file);
end
frames = unique([1:p.viz.frame_skip:numel(res.t), numel(res.t)]);
for k = frames
    if ~ishandle(fig), return; end
    S = robot_shape(res.q(k, :), p.robot);
    set(hB, 'XData', S.body(:, 1), 'YData', S.body(:, 2));
    set(hR, 'XData', S.wheelR(:, 1), 'YData', S.wheelR(:, 2));
    set(hL, 'XData', S.wheelL(:, 1), 'YData', S.wheelL(:, 2));
    set(hH, 'XData', S.head(:, 1), 'YData', S.head(:, 2));
    set(hT, 'XData', res.q(1:k, 1), 'YData', res.q(1:k, 2));
    set(hG, 'XData', res.target(k, 1), 'YData', res.target(k, 2));
    title(ax, sprintf('t = %.2f s \omega_R = %.1f \omega_L = %.1f rad/s', ...
        res.t(k), res.wR(k), res.wL(k)));
    drawnow;
    if ~isempty(gif)
        try
            fr = getframe(fig); [A, map] = rgb2ind(frame2im(fr), 128);
            if k == frames(1)
                imwrite(A, map, gif, 'gif', 'LoopCount', inf, 'DelayTime', 0.03);
            else
                imwrite(A, map, gif, 'gif', 'WriteMode', 'append', 'DelayTime', 0.03);
            end
        catch err
            warning('Salvataggio GIF non riuscito: %s', err.message); gif = '';
        end
    end
    pause(p.viz.pause);
end
end
```

A.36 visualization/plot_environment.m

```
function plot_environment(env, ax)
%PLOT_ENVIRONMENT Disegna bordi, ostacoli e regione proibita (ostacoli gonfiati).
if nargin < 2, ax = gca; end
hold(ax, 'on'); axis(ax, 'equal'); box(ax, 'on');
b = env.bounds;
plot(ax, b([1 2 2 1 1]), b([3 3 4 4 3]), 'k-', 'LineWidth', 1.5);
th = linspace(0, 2*pi, 80);
for j = 1:numel(env.obstacles)
    o = env.obstacles{j};
    if strcmp(o.type, 'circle')
        fill(ax, o.center(1) + o.radius * cos(th), o.center(2) + o.radius * sin(th), ...
            [0.45 0.45 0.5], 'EdgeColor', 'k');
    else
        fill(ax, o.vertices(:, 1), o.vertices(:, 2), [0.45 0.45 0.5], 'EdgeColor', 'k');
    end
end
% contorno dello spazio proibito per il centro del robot (C-ostacoli)
[X, Y] = meshgrid(linspace(b(1), b(2), 200), linspace(b(3), b(4), 200));
C = reshape(clearance([X(:) Y(:)], env), size(X));
contour(ax, X, Y, C, [env.inflation env.inflation], 'LineColor', [0.85 0.4 0.4], 'LineStyle', '--');
xlim(ax, b(1:2)); ylim(ax, b(3:4));
xlabel(ax, 'x [m]'); ylabel(ax, 'y [m]');
```

```
end
```

A.37 visualization/plot_planner_debug.m

```
function fig = plot_planner_debug(env, q0, qf, path, info, p)
%PLOT_PLANNER_DEBUG Mostra la struttura interna del pianificatore.
% cell: celle occupate / libere e nodi ritratti del cammino
% prm : roadmap (nodi e archi)
% rrt : albero di esplorazione
% apf : curve di livello del potenziale, campo -grad U e minimi locali
fig = figure('Name', ['Pianificatore: ' upper(info.planner)], 'Color', 'w');
ax = gca; plot_environment(env, ax);
switch info.planner
    case 'cell'
        g = info.grid; h = g.res / 2;
        [IY, IX] = find(~g.free);
        cx = g.xc(IY); cy = g.yc(IY); cx = cx(:); cy = cy(:);
        n = numel(cx);
        V = [cx - h, cy - h; cx + h, cy - h; cx + h, cy + h; cx - h, cy + h];
        Fc = [(1:n)', (n + 1:2*n)', (2*n + 1:3*n)', (3*n + 1:4*n)'];
        patch(ax, 'Faces', Fc, 'Vertices', V, 'FaceColor', [1 0.8 0.8], 'EdgeColor', 'none');
        [IY, IX] = find(g.free);
        plot(ax, g.xc(IY), g.yc(IY), '.', 'Color', [0.6 0.75 0.6], 'MarkerSize', 4);
        if isfield(info, 'retracted_nodes')
            plot(ax, info.retracted_nodes(:, 1), info.retracted_nodes(:, 2), 's', ...
                'Color', [0 0.5 0], 'MarkerSize', 4);
        end
        title(ax, sprintf('Cell decomposition: %d/%d celle libere', info.n_free, info.n_cells));
    case 'prm'
        V = info.nodes; E = info.edges;
        if ~isempty(E)
            xs = [V(E(:, 1), 1) V(E(:, 2), 1) nan(size(E, 1), 1)]';
            ys = [V(E(:, 1), 2) V(E(:, 2), 2) nan(size(E, 1), 1)]';
            plot(ax, xs(:), ys(:), '-', 'Color', [0.75 0.75 0.9]);
        end
        plot(ax, V(:, 1), V(:, 2), '.', 'Color', [0.2 0.2 0.8], 'MarkerSize', 6);
        title(ax, sprintf('PRM: %d nodi, %d archi, %d tentativi', info.n_nodes, info.n_edges,
            info.attempts));
    case 'rrt'
        V = info.nodes; pr = info.parent; k = find(pr > 0);
        xs = [V(k, 1) V(pr(k), 1) nan(numel(k), 1)]';
        ys = [V(k, 2) V(pr(k), 2) nan(numel(k), 1)]';
        plot(ax, xs(:), ys(:), '-', 'Color', [0.75 0.75 0.9]);
        title(ax, sprintf('RRT: %d nodi in %d iterazioni', info.n_nodes, info.iterations));
    case 'apf'
        b = env.bounds;
        [X, Y] = meshgrid(linspace(b(1), b(2), 120), linspace(b(3), b(4), 120));
        [U, gx, gy] = apf_potential([X(:) Y(:)], qf(1:2), env, p.apf);
        U = reshape(U, size(X));
        contour(ax, X, Y, log(1 + U), 30);
        [Xq, Yq] = meshgrid(linspace(b(1), b(2), 25), linspace(b(3), b(4), 25));
        [~, qx, qy] = apf_potential([Xq(:) Yq(:)], qf(1:2), env, p.apf);
        nq = sqrt(qx.^2 + qy.^2) + eps;
        quiver(ax, Xq(:), Yq(:), -qx ./ nq, -qy ./ nq, 0.4, 'Color', [0.5 0.5 0.5]);
        if isfield(info, 'raw_path')
            plot(ax, info.raw_path(:, 1), info.raw_path(:, 2), '-', 'Color', [0.9 0.6 0],
                'LineWidth', 1);
        end
        if ~isempty(info.local_minima)
            plot(ax, info.local_minima(:, 1), info.local_minima(:, 2), 'mx', ...
                'MarkerSize', 12, 'LineWidth', 2.5);
        end
        title(ax, sprintf('APF: %d minimi locali rilevati, %d fughe', ...
            size(info.local_minima, 1), info.escapes));
end
if ~isempty(path)
    plot(ax, path(:, 1), path(:, 2), 'b-', 'LineWidth', 2);
end
plot_pose(ax, q0, [0 0.7 0], 'q_0');
plot_pose(ax, qf, [0.85 0 0], 'q_f');
end
```

A.38 visualization/plot_pose.m

```
function plot_pose(ax, q, color, label, len)
%PLOT_POSE Disegna una configurazione [x y theta] come punto + freccia.
if nargin < 5, len = 0.6; end
plot(ax, q(1), q(2), 'o', 'MarkerSize', 9, 'MarkerFaceColor', color, 'MarkerEdgeColor', 'k');
quiver(ax, q(1), q(2), len * cos(q(3)), len * sin(q(3)), 0, 'Color', color, ...
    'LineWidth', 2, 'MaxHeadSize', 0.8);
text(ax, q(1) + 0.15, q(2) - 0.3, label, 'FontWeight', 'bold');
end
```

A.39 visualization/plot_results.m

```
function figs = plot_results(res, path, waypoints, q0, qf, env, m, p)
%PLOT_RESULTS Grafici richiesti: traiettoria pianificata ed eseguita, errore
% di tracking, velocita' delle ruote (e v, w), evoluzione dello stato.
figs = [];
% 1) traiettoria pianificata vs eseguita
figs(end+1) = figure('Name', 'Traiettorie', 'Color', 'w'); ax = gca;
plot_environment(env, ax);
h1 = plot(ax, path(:, 1), path(:, 2), 'b--', 'LineWidth', 1.5);
h2 = plot(ax, waypoints(:, 1), waypoints(:, 2), 'b.', 'MarkerSize', 10);
h3 = plot(ax, res.q(:, 1), res.q(:, 2), 'r-', 'LineWidth', 2);
plot_pose(ax, q0, [0 0.7 0], 'q_0'); plot_pose(ax, qf, [0.85 0 0], 'q_f');
legend([h1 h2 h3], {'pianificata', 'waypoint', 'eseguita'}, 'Location', 'southeast');
title(ax, sprintf('%s - traiettoria pianificata ed eseguita', upper(m.planner)));

% 2) errore di tracking
figs(end+1) = figure('Name', 'Errore di tracking', 'Color', 'w');
subplot(2, 1, 1);
plot(res.t, m.cross_track, 'LineWidth', 1.5); grid on;
ylabel('e_{ct} [m]');
title(sprintf('Errore laterale dal percorso (RMS %.3f m, max %.3f m)', m.ct_rms, m.ct_max));
subplot(2, 1, 2);
plot(res.t, res.rho, 'LineWidth', 1.5); grid on;
xlabel('t [s]'); ylabel('\rho [m]');
title('Distanza dal target corrente \rho');

% 3) velocita' delle ruote e del veicolo
figs(end+1) = figure('Name', 'Velocita' ruote', 'Color', 'w');
subplot(2, 1, 1); hold on; grid on;
plot(res.t, res.wR, 'r', 'LineWidth', 1.5); plot(res.t, res.wL, 'b', 'LineWidth', 1.5);
plot(res.t([1 end]), p.robot.wheel_max * [1 1], 'k--');
plot(res.t([1 end]), -p.robot.wheel_max * [1 1], 'k--');
ylabel('\omega [rad/s]'); legend('\omega_R', '\omega_L', 'limite'); title('Velocita' angolari delle ruote');
subplot(2, 1, 2); hold on; grid on;
plot(res.t, res.v, 'LineWidth', 1.5); plot(res.t, res.w, 'LineWidth', 1.5);
xlabel('t [s]'); legend('v [m/s]', '\omega [rad/s]'); title('Velocita' lineare e angolare');

% 4) stato
figs(end+1) = figure('Name', 'Stato', 'Color', 'w');
lab = {'x [m]', 'y [m]', '\theta [rad]'};
for i = 1:3
    subplot(3, 1, i);
    s = res.q(:, i); if i == 3, s = unwrap(s); end
    plot(res.t, s, 'LineWidth', 1.5); grid on; ylabel(lab{i});
end
xlabel('t [s]');

if p.viz.save_figures
    if ~exist(p.viz.output_dir, 'dir'), mkdir(p.viz.output_dir); end
    names = {'traiettorie', 'errore_tracking', 'velocita_ruote', 'stato'};
    for i = 1:numel(figs)
        print(figs(i), fullfile(p.viz.output_dir, sprintf('%s_%s.png', m.planner, names{i})),
            '-dpng', '-r120');
    end
end
end
```

A.40 visualization/robot_shape.m

```
function S = robot_shape(q, robot)
%ROBOT_SHAPE Coordinate del disegno del robot nella configurazione q.
% S.body (cerchio), S.wheelR / S.wheelL (rettangoli), S.head (segmento di prua)
R = [cos(q(3)) -sin(q(3)); sin(q(3)) cos(q(3))];
th = linspace(0, 2*pi, 40)';
body = robot.radius * [cos(th) sin(th)];
w1 = robot.r; ww = 0.4 * robot.r;
wheel = [-w1 -ww; w1 -ww; w1 ww; -w1 ww];
S.body = (R * body)' + q(1:2);
S.wheelR = (R * (wheel + [0 -robot.d/2]))' + q(1:2);
S.wheelL = (R * (wheel + [0 robot.d/2]))' + q(1:2);
S.head = (R * [0 0; robot.radius 0])' + q(1:2);
end
```

A.41 utils/path_length.m

```
function L = path_length(P)
%PATH_LENGTH Lunghezza di una polilinea P (Nx2).
if size(P, 1) < 2, L = 0; return; end
L = sum(sqrt(sum(diff(P, 1, 1).^2, 2)));
end
```

A.42 utils/set_seed.m

```
function set_seed(seed)
%SET_SEED Imposta il generatore casuale (compatibile MATLAB e GNU Octave).
if exist('OCTAVE_VERSION', 'builtin')
    rand('seed', seed); randn('seed', seed); %#ok<RAND>
    rand('state', seed); randn('state', seed); %#ok<RAND>
else
    rng(seed);
end
end
```

A.43 utils/wrap_to_pi.m

```
function a = wrap_to_pi(a)
%WRAP_TO_PI Riporta un angolo nell'intervallo (-pi, pi].
a = mod(a + pi, 2*pi) - pi;
a(a == -pi) = pi;
end
```

A.44 tests/run_all_tests.m

```
function ok = run_all_tests()
%RUN_ALL_TESTS Verifiche automatiche di modello, geometria, pianificatori e controllo.
% >> addpath(genpath(pwd)); run_all_tests
% Restituisce true se tutti i test passano.
p = default_params();
p.viz.show_plots = false; p.viz.animate = false;
p.viz.show_planner_debug = false; p.verbose = false;
rb = p.robot;
n_ok = 0; n_tot = 0;

function check(name, cond)
    n_tot = n_tot + 1;
    if cond, n_ok = n_ok + 1; fprintf(' [PASS] %s\n', name);
    else, fprintf(' [FAIL] %s\n', name); end
end

fprintf('=== Modello cinematico ===\n');
[wR, wL] = unicycle_to_wheels(0.37, -1.1, rb);
[v, w] = wheels_to_unicycle(wR, wL, rb);
```

```

check('conversione (v,w) <-> (wR,wL) invertibile', abs(v - 0.37) < 1e-12 && abs(w + 1.1) < 1e-12);

q = [0 0 0]; om = 5; T = 2; dt = 0.01;
for k = 1:round(T / dt), q = integrate_step(q, om, om, rb, dt, 'rk4'); end
check('ruote uguali -> moto rettilineo, x = r*omega*t', abs(q(1) - rb.*om*T) < 1e-9 && abs(q(2)) <
1e-12 && abs(q(3)) < 1e-12);

q = [1 2 0]; T = 1;
for k = 1:round(T / dt), q = integrate_step(q, om, -om, rb, dt, 'rk4'); end
check('ruote opposte -> rotazione pura, theta = 2 r omega t / d', ...
norm(q(1:2) - [1 2]) < 1e-12 && abs(q(3) - wrap_to_pi(2*rb.*om*T/rb.d)) < 1e-9);

qe = integrate_step([0.3 -0.2 0.4], 12, 7, rb, 0.05, 'exact');
qr = integrate_step([0.3 -0.2 0.4], 12, 7, rb, 0.05, 'rk4');
check('RK4 coincide con la soluzione esatta (errore < 1e-8)', norm(qe - qr) < 1e-8);

[a, b, s] = saturate_wheels(40, 10, 20);
check('saturazione: limite rispettato e curvatura preservata', abs(a - 20) < 1e-12 && abs(b - 5) <
1e-12 && s == 0.5);

fprintf('=== Geometria ===\n');
env.bounds = [0 10 0 10]; env.inflation = 0.25; env.robot_radius = 0.2;
env.obstacles = {make_obstacle('circle', [5 5], 1), make_obstacle('polygon', [1 1; 2 1; 2 2; 1 2])};
D = obstacle_distances([7 5; 1.5 1.5; 3 1.5], env);
check('distanza da un cerchio', abs(D(1, 1) - 1) < 1e-12);
check('distanza negativa dentro un poligono', abs(D(2, 2) + 0.5) < 1e-12);
check('distanza esterna da un poligono', abs(D(3, 2) - 1) < 1e-12);
check('segmento che attraversa un ostacolo non e'' libero', ~is_segment_free([3 5], [7 5], env,
0.02));
check('segmento lontano dagli ostacoli e'' libero', is_segment_free([3 8], [7 8], env, 0.02));

fprintf('=== Pianificatori (scenario base) ===\n');
for pl = {'cell', 'prm', 'rrt', 'apf'}
    pp = p; pp.planner = pl{1};
    [env2, q0, qf] = build_environment(pp);
    [path, wp, info] = plan_path(env2, q0, qf, pp);
    good = info.success && is_path_free(path, env2, 0.01) && is_path_free(wp, env2, 0.01) ...
        && norm(path(1, :) - q0(1:2)) < 1e-9 && norm(path(end, :) - qf(1:2)) < 1e-9;
    check(sprintf('%s: percorso q0 -> qf collision-free', upper(pl{1})), good);
end

fprintf('=== Minimi locali APF (scenario trap) ===\n');
pp = p; pp.planner = 'apf'; pp.env.scenario = 'trap'; pp.apf.escape = false;
[env3, q0, qf] = build_environment(pp);
[~, ~, info] = plan_path(env3, q0, qf, pp);
check('APF senza fuga si blocca nel minimo locale della U', ~info.success &&
~isempty(info.local_minima));
pp.apf.escape = true;
[path, ~, info] = plan_path(env3, q0, qf, pp);
check('APF con random walk esce dal minimo locale', info.success && is_path_free(path, env3, 0.01));

fprintf('=== Controllo e simulazione ===\n');
for mode = {'waypoints', 'continuous'}
    pp = p; pp.ctrl.ref_mode = mode{1};
    out = run_project(pp);
    m = out.metrics;
    check(sprintf('controllo %s: goal raggiunto senza collisioni', mode{1}), ...
        out.success && m.final_pos_error < pp.ctrl.goal_tol && ...
        m.final_head_error < pp.ctrl.heading_tol && m.max_wheel <= rb.wheel_max + 1e-9);
end

fprintf('\n%d / %d test superati\n', n_ok, n_tot);
ok = n_ok == n_tot;
end

```